

Algorithms

Reference catalogue of the **DSP, mathematical, graphics, and signal-processing algorithms** implemented across Phonyser's four measurement modules — the **Signal Generator, Oscilloscope, FFT Analyzer, and Frequency Response** views. For each algorithm it gives a short description of *what it does* and *how it is used in the app*, with `file:line` citations into the source as it stood when this document was generated.

This is generated by reading the code and its comments; it documents what is **actually implemented** (including paths that are wired but currently disabled, flagged as such), not an aspirational design.

Named-algorithm index

The eponymous / textbook algorithms used in the code, with where to read more. The detailed per-module sections below carry the math and `file:line` citations.

Signal synthesis & quantization

Algorithm	What it is	Where
Direct Digital Synthesis (DDS)	64-bit phase-accumulator oscillator; exact mod- 2^{64} phase, phase-continuous on frequency change	\$1.1
Angle-sum + Taylor sine	4096-entry table for $\theta_0 + 5$ th-order Taylor <code>sin/cos($\Delta\theta$)</code> reconstruction (<code>sin($\theta_0+\Delta\theta$)=...</code>)	\$1.2
Farina exponential sine sweep (ESS)	log-chirp <code>sin(K(e^{t/L}-1))</code> ; reused byte-identical as the deconvolution reference	\$1.7, \$4.1
Voss-McCartney pink noise	summed-octave 1/f generator, $O(1)$ /sample, -3 dB/oct	\$1.9
TPDF dither	triangular-PDF (difference of two uniforms) dither before quantization, decorrelates LSB error	\$1.13
Hann / raised-cosine taper	sweep fade-in/out and window leakage suppression	\$1.8, \$4.1

Spectral / FFT

Algorithm	What it is	Where
Cooley-Tukey radix-2 FFT (Danielson-Lanczos lemma)	in-place decimation-in-time FFT; the D-L lemma is the recursive even/odd halving it implements	\$3.1
Bit-reversal permutation	index-reversal reorder before the butterfly stages	\$3.1
Goertzel single-bin DFT	$O(N)$ magnitude/phase at one frequency without a full FFT — the engine for measuring the exact fundamental frequency (sub-bin) in both Scope and FFT; a full FFT only resolves to the nearest bin	\$2.2, \$2.5, \$3.5
Blackman-Harris (4-term)	low-sidelobe analysis window	\$3.2
Nuttall (7-term)	very-low-sidelobe window (BH7)	\$3.2

Blackman-Harris)		
Flat-top (SR785 5-term)	amplitude-accurate window	§3.2
HFT144D / HFT248D (Heinzel high-flattop)	very-low-leakage amplitude-flat windows for high-dynamic-range tone measurement	§3.2
Kaiser window (KB24 / KB38)	I_0 -based window at $\beta=24$ / $\beta=38$ — ≈ -226 dB / below-double-precision sidelobes	§3.2
Dolph-Chebyshev window	equiripple window at 150–300 dB sidelobes, built via inverse-DFT of $T_{\lfloor N/2 \rfloor}$	§3.4
Chebyshev polynomial T_n	trig/hyperbolic form, drives the Dolph-Chebyshev window	§3.2, §3.4
Parabolic / quadratic peak interpolation	3-point sub-bin peak/frequency refinement	§2.2, §3.5, §3.11, §4.3
Quinn/Jacobsen-style phase-difference estimator	sub-bin frequency from inter-frame phase advance	§3.5
Smith & Serra interpolated peak	quadratic peak-value estimate for IMD tone levels	§3.11
Coherent (vector) averaging + per-lobe de-rotation	phase-aligned complex sum; constant-phase-per-lobe (not a per-bin ramp)	§3.6, §3.14
CCIF/DIN intermodulation products	difference-frequency IMD product grid & ratios	§3.11

Filtering & estimation

Algorithm	What it is	Where
Lanczos (windowed-sinc) interpolation	Whittaker-Shannon reconstruction, $A=16$ lobes, anti-alias-scaled kernel	§2.4
Schmitt trigger (hysteresis)	dual-threshold edge qualification for scope triggering	§2.1
Chebyshev Type I IIR low-pass	biquad-cascade 80 kHz HF filter (note: stale "Butterworth" doc comment)	§2.5
Bilinear transform + pre-warping	analog→digital pole mapping $\tan(\pi \cdot f_c / f_s)$	§2.5
Median (de-spike) filter	sliding-window median, edge-preserving impulse removal	§2.5
IIR feedback comb	$(1-z^{-N}) / (1-\alpha \cdot z^{-N})$ mains-harmonic notch comb	§2.5, §3.15
LMS line canceller	adaptive sinusoid-model mains suppression; per-harmonic quadrature weights, Rodrigues-rotated references, no DC term	§2.5
Synchronous template subtraction	period-locked 512-bin mains-cycle template, LMS-learned, preserves non-periodic tones	§2.5
Mains 50/60 Hz auto-	Goertzel dual-band scan + parabolic sub-step + EWMA lock	§2.5

detection		
Synchronous I/Q (lock-in) detection	quadrature correlation to recover the dual-tone beat phase	\$2.6
Frequency-domain deconvolution $H=Y/X$	full transfer function (mag+phase) from one FFT pair	\$4.3
Looping circular-transform deconvolution	power-of-two-period FFT $\Rightarrow$ shift-invariant $\ H\ $; live tune-notch tracking with no trigger	\$4.11
Linear-phase delay rotation	DAC $\leftrightarrow$ ADC transport-delay removal via $e^{j2\pi k\delta/M}$	\$4.3
Impulse-response time-gating	Hann-gated IR to trade resolution for less ripple (off by default)	\$4.3
RIAA phono EQ	3180/318/75 μ s (+IEC 7950 μ s) time-constant network	\$4.7

Statistics, smoothing & graphics

Algorithm	What it is	Where
Welford's online variance	one-pass mean/ σ for live measurement stats	\$2.3
Median + MAD robust floor	$\text{median} + k \cdot \text{MAD}$ noise-floor / outlier rejection	\$3.10, \$3.12, \$3.13
Savitzky-Golay smoothing	least-squares polynomial sliding filter (peak-preserving)	\$4.4
Gauss-Jordan inversion	solves the Savitzky-Golay Vandermonde coefficients	\$4.4
Exponential moving average (EMA)	smooths live-meter RMS / mains-lock detections	\$2.10, \$4.9
Liang-Barsky line clipping	per-segment polyline clip so far-off coords don't wrap (GDI $\pm 32k$)	\$3.16
1-2-5 decade "nice number" ticks	log/linear axis major/minor tick generation	\$2.8, \$3.16
Min/max column decimation	per-pixel-column envelope bars for dense waveforms /spectra	\$2.9, \$3.16
Digital-phosphor persistence	RGBA16F framebuffer with exponential $\exp(-dt/\tau)$ afterglow decay (GL path)	\$2.11

ADC characterisation

Algorithm	What it is	Where
Code-density histogram	per-code hit-count linearity test; 2^{bitDepth} bins, chunked 32-bit storage	\$5.1
Arcsine (sine-wave) PDF reference	ideal code density $1/(\pi\sqrt{A^2-V^2})$; CDF-inversion amplitude estimate	\$5.1
Moving-average / FIR density reference	12/25-tap FIR-smoothed count as the ideal, outlier-trimmed	\$5.1
DNL / INL code-density test	$\text{DNL}=\text{weight}-1$, $\text{INL}=\sum \text{DNL}$ in LSB	\$5.2
Least-squares sine-fit calibration	golden-section frequency + 3×3 normal-equation sine fit $\rightarrow$ per-code error	\$5.3

Conventions & shared primitives

A few facts recur throughout; they are stated once here:

- **Normalized sample domain.** Audio samples live in floating point in the range `[-1, +1]` everywhere internally. Conversion to/from PCM integers happens only at the device/file boundary.
- **Voltage scaling.** A normalized sample is turned into volts with `peakVolts = adcFsVoltageRms * √2` (ADC full-scale), and the generator's output scales against the calibrated DAC full-scale RMS voltage (`dacFsVoltageRms`, a preference set by the Calibrate-DAC dialog and constructor-injected into `SignalGenerator`). This keeps a captured file and the live trace reading the same amplitude.
- **Goertzel single-bin DFT** (`coeff = 2 · cos(2πf/fs)`, magnitude $\sqrt{q1^2 + q2^2 - coeff \cdot q1 \cdot q2}$) is the workhorse for measuring one frequency's magnitude without a full FFT; it appears in the scope frequency meter, the mains-comb tracker, and the FFT κ-refit.
- **Lanczos (windowed-sinc) reconstruction** is used **only** in the oscilloscope's time-domain view (`ScopeLanczos`); the frequency-response path resamples its stored curves by log-frequency interpolation, not Lanczos.
- **Shared frequency-domain plotting.** The FFT and Frequency-Response views share one chart engine (`AbstractMeasurementView` / `AbstractFreqDomainView`) for log/linear "nice" axis ticks, value→pixel mapping, and clipped polyline rendering. It is documented once in [§3.16](#).

Contents

1. [Signal Generator](#) — DDS synthesis, waveform math, sweeps, FFT-bin snapping, dither/PCM, output backends
 2. [Oscilloscope](#) — triggering, raw-signal frequency measurement, Lanczos reconstruction, DSP filters (comb / LMS / sync-subtract mains), beat reconstruction, rendering, digital-phosphor persistence
 3. [FFT Analyzer](#) — radix-2 FFT, windows, coherent averaging & phase-lock, THD/IMD/noise floor, lobe stretch, discontinuity rejection, mains suppression, shared plotting
 4. [Frequency Response](#) — Farina log-sweep deconvolution, Savitzky-Golay smoothing, `.frc` calibration, RIAA EQ, compare/diff, live metering, tune-notch live measurement
 5. [ADC Characterisation](#) — code-density histogram (arcsine / moving-average PDF), DNL/INL, regression sine-fit calibration
-

1. Signal Generator

The signal generator synthesizes test waveforms sample-by-sample in floating point, packs them to PCM, and streams them to the DAC through one of three audio backends. The core is `SignalGenerator.java`; GUI wiring lives in `GeneratorController/GeneratorPane`; PCM encoding and device I/O live in the per-backend `*Generator` classes and the file exporters.

1.1 Direct Digital Synthesis (phase-accumulator oscillator)

All periodic forms are driven by a **64-bit phase accumulator** held in a `long`, advanced by a fixed `phaseInc` each sample (`SignalGenerator.java`, `nextSample()`).

- **Phase increment:** `phaseInc = round(f / fs * 264)` (`toPhaseInc`, used by the constructors and `setFrequency`). One full turn equals 2⁶⁴, so plain `long` overflow IS the modulo-2⁶⁴ phase

wrap — no masking, no per-sample drift. The increment is computed in a `double`, whose 53-bit mantissa caps the useful resolution at $f_s/2^{53} \approx 43$ pHz at 384 kHz — the point of the width is headroom for the frequency-lock loop: sub-ppb trim corrections stay representable instead of quantizing away. `phaseInc` is `volatile` so GUI-thread frequency edits reach the audio thread atomically and **phase-continuously** — the accumulator is preserved across a frequency change, so there is no click.

- **Live form/frequency swaps** keep the accumulator, so transitions between sine/triangle/rectangle are smooth (`setForm`).
- A **second accumulator** `phaseAcc2/phaseInc2` exists solely for `DUAL_TONE` and is advanced only when that form is active.

1.2 Sine synthesis — table lookup + Taylor correction

`ddsSine()` reads a **4096-entry** (`TABLE_BITS=12`) full-wave `SINE_TABLE/COS_TABLE` built at class load. The top 12 bits of the accumulator select the coarse angle θ_0 ; the remaining 52 low bits form a fractional phase error $\Delta\theta$ (radians, via `TWO_PI_OVER_2_64`). The exact sample is reconstructed with the angle-sum identity:

$$\sin(\theta_0 + \Delta\theta) = \sin(\theta_0) \cdot \cos(\Delta\theta) + \cos(\theta_0) \cdot \sin(\Delta\theta)$$

where `cos( $\Delta\theta$ )` and `sin( $\Delta\theta$ )` are **5th-order Taylor series** evaluated in Horner form (`cos` $\approx 1 - \Delta\theta^2/2! + \Delta\theta^4/4!$, `sin` $\approx \Delta\theta - \Delta\theta^3/3! + \Delta\theta^5/5!$). With $\Delta\theta$ bounded by one table step (≈ 1.53 mrad), the truncation error $\sim \Delta\theta^7/7! \approx 1e-24$ lies far below double precision. `ddsSine2()` is a byte-identical copy reading `phaseAcc2`, kept separate so the JIT inlines the dual-tone per-tone read without a parameter copy.

1.3 Rectangle, triangle and sawtooth (piecewise-linear, exact)

These map the accumulator to a normalized phase `frac` $\in [0,1)$ and are exact by construction (no Taylor term needed):

- **Rectangle / pulse** (`ddsRectangle :810`): `+1` while `frac < rectDuty`, else `-1`. Duty $\neq 0.5$ deliberately yields a non-zero DC component for pulse test signals. Duty clamped to `[0.001, 0.999]` so the train never collapses to DC (`setRectangleDuty :519`).
- **Triangle with adjustable duty** (`ddsTriangle :823`): rising ramp `-1 $\rightarrow$ +1` over the first `triDuty` fraction, falling ramp `+1 $\rightarrow$ -1` over the remainder. `triDuty=0.5` is a symmetric triangle; values near 1.0/0.0 approach sawtooth / inverted-sawtooth. Clamped `[0.001, 0.999]` (`setTriangleDuty :532`).

1.4 Dual-tone (IMD test signal)

`DUAL_TONE` sums two independent DDS sines: `ddsSine()·w1 + ddsSine2()·w2` (`:554`). Per-tone linear weights `dualW1`, `dualW2` are each `amplitude_percent/100`, clamped `[0,1]` independently (`setDualToneAmplitudes :494`). The two uncorrelated sines give combined $\text{RMS} = \sqrt{(w_1^2 + w_2^2)/2}$ (`rawRmsDualTone :426`), and the overall `amplitude` scale is **re-normalized against that RMS** every time the split or `Vrms` changes, so the combined output `Vrms` always equals the `Amplitude` field regardless of the split (a 50/50 split is not quieter than 100/0). The divisor is floored at `1e-12` to avoid NaN when both tones are 0 %. Tone 2 is FFT-bin-snapped independently of tone 1 (see 1.10).

1.5 Harmonic-compensated sine (pre-distortion)

SINE_COMPENSATED (ddsSineCompensated) subtracts measured distortion harmonics from a clean DDS sine so the analog chain's residual H2..Hn cancel. Each harmonic's phase is generated by **integer-multiplying the master accumulator** by the harmonic number `h`: `hPhase = phaseAcc·hNum + hPhase0ff64` (wraps mod 2^{64} ; the per-harmonic initial phases are pre-converted to 64-bit phase offsets on first use). This locks the correction tone exactly to `h·f_DDS` — no drift or beating against the real distortion — and the correction `s -= hAmpRatio[i]·ddsCos(hPhase)` runs on the same table+Taylor kernel as the fundamental (`ddsCos`), so no `Math.cos` sits on the per-sample path (~2 M trig evaluations/s at 5 harmonics × 384 kHz).

System-delay phase calibration (`loadHarmonics :925`, `loadHarmonicsFromResult :864`): a DDS sine at $n=0$ has FFT phase $-\pi/2$, so the measured fundamental phase implies a loop delay $\omega D = -(\phi_1 + \pi/2)$. This is converted to rad/Hz (`delayRadPerHz =  $\omega D / f_{\text{fund}}$`) and each harmonic's emit phase is `$\phi_{\text{init}} = \phi_{\text{h\_measured}} + f_{\text{h}} \cdot \text{delayRadPerHz}$` , so the correction arrives at the ADC with the phase that nulls the measured harmonic. Amplitudes are stored as ratios to the fundamental (`amplitude_pct/100`) so they are level-independent. Corrections load from `fft_harmonics_*.csv`, an `applied_compensation_*.csv`, directly from an `FftResult`, or from pre-accumulated complex corrections (iterative workflow). Incoherent FFT results carry no phase, so phase correction is disabled with a warning and amplitude-only correction applied (`:865`). CSV phase is taken from `re/im` columns at full precision when present, else from `phase_deg`. If a CSV's calibration amplitude differs from the requested Vrms, per-harmonic ratios are rescaled by `csv_amplitude/amplitudeVrms`.

1.6 Linear (constant-rate) frequency sweep / chirp

LINEAR_SWEEP (`linearSweepNext :574`) is a phase-continuous chirp whose **instantaneous frequency ramps linearly** $f_0 \rightarrow f_1$ across `sweepPeriodSamples`:

```
instF = f0 + (f1-f0)·(sweepIdx/period)
s      = sin(sweepPhase);  sweepPhase += 2π·instF/fs
```

Phase is accumulated in a `double` and wrapped modulo 2π to bound magnitude (`:585`). It restarts phase-continuously each cycle when `sweepLoop` is true, else goes silent after one cycle. A **Hann fade** envelope can be applied per cycle (see 1.8).

1.7 Logarithmic (Farina) exponential sine sweep + deconvolution reference

LOG_SWEEP is a Farina exponential chirp, **pre-rendered** at unit amplitude into `logSweepBuffer` (`renderLogSweep :367`):

$$x(t) = \sin(K \cdot (\exp(t/L) - 1)), \quad L = T / \ln(f_1/f_0), \quad K = 2\pi \cdot f_0 \cdot L$$

Instantaneous frequency advances log-linearly $f_0@t=0 \rightarrow f_1@t=T$; sample 0 is exactly 0 so it concatenates with leading silence without a discontinuity. The buffer is exposed via `getLogSweepBuffer()` so the frequency-response analyzer reuses the **byte-identical reference X(t)** for FFT deconvolution $Y(f)/X(f) \rightarrow H(f)$. Playback (`logSweepNext :600`) emits `logSweepLeadIn` zeros, replays the buffer, and on loop wraps back to the buffer start (skipping the lead-in). Live edits to start/stop/duration re-render the buffer (`rebuildLogSweepBuffer :697`, `setSweepDurationSamples :676`).

1.8 Hann fade-in / fade-out envelope

`sweepEnvelope()` (:625) applies a raised-cosine (Hann) taper at the edges of each sweep cycle: fade-in $0.5 \cdot (1 - \cos(\pi \cdot \text{idx} / \text{fadeIn}))$ over the first `fadeInSamples`, a mirrored fade-out over the last `fadeOutSamples`, and `1.0` in the steady-state middle. Fade lengths are clamped to half the cycle so in- and out-fades cannot overlap (`setSweepParams` :642).

1.9 Noise generators (white, Voss-McCartney pink)

- **White noise** (:548): `rng.nextGaussian()` — Gaussian, $\sigma=1$.
- **Pink noise** (`pinkNoise` :843): the **Voss-McCartney** summed-octave algorithm with `PINK_OCTAVES=16` rows plus one always-updated white term. On each tick the row indexed by the number of trailing zero bits of a counter is refreshed and a running sum maintained, giving an $O(1)$ per-sample ≈ -3 dB/octave ($1/f$) spectrum. Two variants: the **Gaussian** source (`PINK_NOISE`) and a **uniform / flat-amplitude** source `rng.nextDouble() * 2 - 1` (`PINK_NOISE_LINEAR`).

1.10 RMS normalization (V RMS → linear peak amplitude)

A target output in **volts RMS** is converted to the internal linear peak scale by `amplitude = Vrms / (dacFsVoltageRms * rawRms(form))` (constructors, `setAmplitudeVrms`). `rawRms()` returns each waveform's analytical unit-amplitude RMS: sine/sweep $1/\sqrt{2}$, triangle $1/\sqrt{3}$ (duty-independent), rectangle `1` (any duty), white noise `1`, pink $1/\sqrt{N+1}$ (Gaussian) or $1/\sqrt{3(N+1)}$ (uniform), dual-tone $\sqrt{((w_1^2 + w_2^2) / 2)}$. `dacFsVoltageRms` is the **calibrated DAC full-scale RMS voltage** — a preference set by the Calibrate-DAC dialog, constructor-injected into the generator (it never reaches into Preferences itself) and pushed live via `setDacFsVoltageRms` on recalibration, so a running tone re-scales without restart. Because peak/RMS ratio differs per form, switching forms re-divides the cached `currentVrms` so the output `Vrms` is held constant (`setForm`).

1.11 FFT-bin snapping (coherent-frequency rounding)

`FftBinSnap.snapIfEnabled()` rounds a requested frequency to the nearest FFT bin centre so a tone lands exactly on a bin (avoids spectral leakage in coherent FFT analysis): `binHz = fs/fftLength`; `snapped = round(raw/binHz) * binHz`. It only fires for `SINE` and `DUAL_TONE` when the user enabled snap-to-bin; all other forms pass through unchanged. Each dual-tone tone is snapped independently. The snapped value is what the DDS actually emits while the `raw` entered value is what's persisted (`GeneratorController.java:98`, :183). The FFT view re-publishes a new FFT length so the running tone re-snaps live, and a frequency-lock loop publishes sub-Hz trims (`GENERATOR_FREQ_TRIM`) that are live-applied to the DDS.

1.12 Sweep-state reset after JIT warmup

`resetSweepPosition()` (:388) zeroes `logSweepIdx`, `sweepIdx`, `sweepPhase`, and `phaseAcc`. Every backend calls it **after** a pre-stream JIT warmup that consumed `nextSample()` calls — otherwise a frequency-response measurement would capture a sweep that started mid-buffer while the deconvolution reference starts at zero, breaking alignment.

1.13 Float → PCM conversion: TPDF dither + quantization

PCM packing is centralized in `PcmQuantizer` (`sound/PcmQuantizer.java`), owned by every playback backend (`JavaSoundGenerator`, `WasapiGenerator`, `WdmksGenerator`) so the streamed bytes are identical regardless of path; the file exporters use the same dither/quantize math:

- **TPDF dither:** `tpdfNoise = (rng.nextDouble() - rng.nextDouble()) / 2^(ditherBits-1)`. The difference of two uniform randoms is a triangular PDF spanning ± 1 LSB at the chosen bit depth; added before quantization it decorrelates quantization error. `ditherBits=0` disables it; the count is live-tunable (volatile, read per sample) and capped at the output bit depth. The RNG is a render-thread-confined `SplittableRandom` (no per-sample monitor/CAS cost).
- **Clamp + quantize:** sample is clamped to `[-1,1]`, then `pcm = round(sample * (2^(N-1)-1))` written **little-endian**; 8-bit is **signed** (`round(s*127)`) — all backends open their lines /streams in signed formats.
- **Channel interleaving:** stereo, **both channels carry the same sample** (mono signal duplicated L/R).
- The synthesis layer stays dither-free, so exports and the deconvolution reference $X(t)$ remain the ideal waveform; dither exists only at the device boundary where its ± 1 LSB amplitude is defined by the target depth.

1.14 File export (WAV / AIFF / FLAC)

`SignalFileExporter` picks the container by extension (`.wav/.flac/.aif/.aiff`) and writes through `WavWriter/AiffWriter/FlacWriter` behind a common `PcmSink`, using the same `fillBuffer` dither /quantize path as live playback. For **periodic** forms it truncates the length to the largest integer multiple of one signal period (`truncateToFullPeriods :95, period = round(fs/freq)`) so the file loops click-free; noise/sweeps use the requested duration as-is. `WavSignalExporter` is the WAV-only legacy variant. 8-bit export is WAV-only (FLAC rejects 8-bit).

1.15 Per-backend output mechanics

Three backends implement `AudioPlayback`; the GUI/CLI drive them identically through `AudioBackend`. All do a **~500 ms JIT warmup** of the encode hot path before the device pulls real audio (so C2 has compiled `nextSample/encode` and deopts have settled), then `resetSweepPosition()`.

- **JavaSound** (`JavaSoundGenerator`): blocking `SourceDataLine.write()` streaming in 4096-frame chunks; pre-fills the hardware buffer before signalling ready. The realtime scheduling is offloaded to the native mixer (zero JNA round-trips), which the comments call the gap-free path. (This is the preferred path: a direct WASAPI render loop periodically inserts a one-period flatline because it does five JNA round-trips per tick without MMCSS registration — routing DDS through JavaSound's `SourceDataLine` avoids it.)
- **WASAPI** (`WasapiGenerator`): exclusive **event-callback** mode (shared fallback), pulling via `IAudioRenderClient::GetBuffer/ReleaseBuffer` on each event-handle wake. Pre-fills the whole HW buffer before `Start` (else exclusive mode glitches on the first tick). A render-loop supervisor tracks tick interval vs an `expectedTickNanos*1.2` underrun threshold and a `½·tick` fill budget, logging late ticks / slow fills / partial fills every 5 s. Encodes into a reusable native scratch buffer (no hot-path allocation). Considered a legacy path (see the note on JavaSound above).
- **WDM-KS** (`WdmksGenerator`): PortAudio WDM-KS **callback mode** — `paCallback` fills the output pointer on PortAudio's realtime thread, returning `paContinue/paComplete` based on stop-flag and frame count, and counts `paOutputUnderflow/paOutputOverflow` flags (drained into rate-limited warnings). Uses the device's `defaultHighOutputLatency` and `paFramesPerBufferUnspecified` so the KS pin picks its natural size (a 0 latency or fought buffer size stalls WDM-KS after the priming callback).

The generator playback thread runs at `Thread.MAX_PRIORITY` to keep exclusive-mode backends ahead of GC/GUI and avoid mid-plateau dips (`GeneratorController.java:220`).

1.16 File playback ("Load from...")

`FilePlayController` decodes a chosen WAV/AIFF/FLAC (FLAC via jflac SPI) through `javax.sound.sampled` and streams the decoded PCM to a default `SourceDataLine`, reopening the stream from the file on EOF when looping. It is intentionally **not** routed through `AudioBackend` — it is a monitoring convenience, not measurement-grade output.

1.17 GUI numeric / unit algorithms

- **Amplitude units** (`AmplitudeUnit`): all conversions canonicalize through V RMS. $\text{dBV } V_{\text{rms}} = 10^{(\text{dBV}/20)}$; $\text{dBFS } V_{\text{rms}} = f_s \cdot 10^{(\text{dBFS}/20)}$ using the ADC full-scale RMS; inverse uses $20 \cdot \log_{10}(\cdot)$ floored at $1\text{e-}12$ to avoid $-\infty$. $\mu\text{V}/\text{mV}/\text{V}$ are pure metric scalings with an auto-rescale group.
- **Duty step-by-sample** (`stepDutyBySamples` `GeneratorPane.java:1271`): the duty arrow/wheel step moves the high/rise edge by exactly **one sample of the current period** ($n = \text{round}(f_s/\text{freq})$), converting $\text{duty}\% \leftrightarrow \text{sample count } k$ and clamping $k \in [[0.01n], [0.99n]]$, so the duty grid tracks the integer-sample resolution of the waveform.
- **Frequency stepping**: wheel = $\pm 5\%$ multiplicative, arrows = ± 1 Hz additive, floored at 0.01 Hz.

1.18 Waveform pictograms (SWT vector drawing)

`SignalFormIcon` renders a 24×24 transparent-ARGB pictogram per form with antialiased SWT `Path`/`drawLine` primitives, cached per `Display`. Notable math: the dual-tone glyph draws $\sin(5\pi t) + \sin(6\pi t)$ to show the beat envelope; the sweep glyphs use phase $\propto t^2$ (linear) vs $\propto \exp(3t) - 1$ (log) to visually shrink the period left→right; noise glyphs use a seeded `Random` (deterministic per variant) with 2-/3-pass neighbour averaging to distinguish white from pink.

2. Oscilloscope

The oscilloscope module captures soundcard audio into a ring buffer, renders a triggered waveform with sub-sample anchoring, and computes live time/amplitude measurements off a background worker. Rendering runs at the SWT paint rate ($\sim 60+$ Hz); measurements run on a separate 10 Hz worker thread. Throughout, normalized samples live in $[-1, +1]$ and are scaled to volts by $\text{peakVolts} = \text{adcFsVoltageRms} \cdot \sqrt{2}$ (e.g. `ScopeView.java:2360`).

2.1 Triggering (edge detection, hysteresis, sub-sample refinement)

Hysteresis-banded Schmitt trigger — `ScopeTrigger.find` (`ScopeTrigger.java:70`). Walks `data` [from..to) with two thresholds $l_o = \text{level} - \text{hysteresis}$, $h_i = \text{level} + \text{hysteresis}$. A rising trigger is *qualified* only when the signal, having previously dipped below l_o (Schmitt state -1), records a `level` crossing and then rises clear above h_i ; the symmetric rule applies to falling edges. The incoming Schmitt state is seeded by scanning backwards from `from` to the first sample firmly outside the dead-band (`ScopeTrigger.java:79`). With `hysteresis == 0` the two thresholds collapse onto `level`, recovering plain single-sample-bracket triggering. The function returns the **rightmost** (most recent) qualified crossing's fractional index, or -1 . Hysteresis is exposed in screen units: $\text{hysteresisDiv} \cdot \text{vDiv} / \text{peakVolts}$ converts a divisions-of-screen setting into normalized sample units (`ScopeView.java:1956`).

Holdoff / minimum spacing — the second `find` overload (`ScopeTrigger.java:70`) rejects a qualified crossing closer than `minSpacingSamples` to the last accepted one. Used in dual-tone mode to keep one trigger per $|F1-F2|$ beat (`minSpacingSamples = sampleRate / |F1-F2|`).

Sub-sample crossing refinement — two methods:

- *Linear* (`ScopeTrigger.linear`, `ScopeTrigger.java:135`): `prevIdx + (level - prev)/(curr - prev)`.
- *Sinc bisection* (`ScopeTrigger.refine`, `ScopeTrigger.java:148`): 10-iteration bisection of the band-limited reconstruction `ScopeLanczos.lanczos(data, n, m, 1.0)` between the bracketing samples, giving $2^{-10} \approx 0.001$ -sample precision. The trigger channel's own sinc-interp preference selects which is used (`ScopeView.java:1922`).

Trigger level computation — the level slider is a fraction of canvas height; it is mapped to a normalized sample value via `(triggerCenterY - triggerLevelY) / vScaleTrig` where `vScaleTrig = peakVolts/vDiv * pixelsPerDivY` (`ScopeView.java:1931`). In AC-coupled mode the displayed trace has its DC bias removed, so the channel's averaged DC mean is *added back* into the threshold so the comparator fires where the user sees the dashed level line (`ScopeView.java:1951`).

Modes (`TriggerMode: AUTO / NORMAL / SINGLE`, `TriggerMode.java`; `TriggerEdge: RISE / FALL`, `TriggerEdge.java`). The three-step anchoring logic (`ScopeView.java:2017–2088`): (1) if a fresh trigger is found, anchor the window so the centre pixel maps exactly to `triggerFrac`, splitting the fractional window-left edge into `floor(dispStart) + fraction(subSampleOffset)` to keep odd-length windows centred without a 0.5-sample bias; (2) else hold the previous trigger if its absolute sample position is still in-buffer; (3) else AUTO/SINGLE free-run on the latest samples, while NORMAL leaves the pane blank. The trigger position is stored as an *absolute* `writePos` (`lastTriggerAbsPos`) so the same time stays centred across redraws as the ring keeps scrolling (`ScopeView.java:2033`). SINGLE freezes the frame into dedicated `capturedLeft/Right` buffers (`captureSingleFrame`, `ScopeView.java:2307`), independent of the ring so it survives indefinitely.

Pre/post-roll sizing — the read window is `2*displaySamples + 2*LANCZOS_PADDING + extraLookback` (`ScopeView.java:1852`), guaranteeing a full display window either side of any trigger-position-slider setting plus an extra lookback (up to `MEAS_MAX_SAMPLES`) so low-frequency signals still contain an edge. The trigger search range is clamped asymmetrically by `triggerPosFrac` so the resulting display window stays inside the buffer (`ScopeView.java:1914`).

2.2 Fundamental-frequency measurement (raw signal, comb-seeded)

Frequency is measured in `SignalMeasurements.compute` (`SignalMeasurements.java:72`) using a **crossing-seed → Goertzel-refine** strategy, deliberately off the *raw* (un-comb-filtered) signal:

1. **Coarse period from threshold crossings** — count rising crossings of the half-amplitude midpoint `(min+max)/2` (`SignalMeasurements.java:102`); the two outermost crossings span an integer number of cycles, giving `coarsePeriod = (lastCross - firstCross)/(crossCount - 1)` (`SignalMeasurements.java:166`). Crossing-derived periods track the true fundamental directly, so a narrow-duty rectangle yields the right answer rather than locking onto a near-equal-amplitude harmonic (per the comment at `:160`).
2. **Three-stage Goertzel refinement** — `refineAroundEstimate` (`SignalMeasurements.java:229`): a coarse scan (100 probes over the half-width), then two successive scans each shrinking the step by 20×, then **parabolic interpolation** across three adjacent fine bins (`:264`) for sub-step precision ($\leq \sim 0.005$ Hz on a 1 s buffer), at a fixed ≈ 180 Goertzel evaluations.

3. **Quality gate** — the lock ratio `mag/n/rmsAc` (≈ 0.707 for a pure sine, ~ 0.005 for white noise) must clear 0.1 to admit rectangles down to $\sim 1\%$ duty (`SignalMeasurements.java:193`).
4. **Broad-band fallback** — when crossings look unreliable (ratio < 0.1 , i.e. SNR $< \sim 10$ dB), `scanForFundamental` (`SignalMeasurements.java:293`) sweeps the whole band on a 0.1 s sub-window at native FFT-bin resolution, then refines the peak.
5. **Leakage-debiased final estimate** — the chosen peak is re-refined on a **Hann-windowed** copy (`hannWindowed`, `SignalMeasurements.java:334`); the window's low side-lobes suppress the negative-frequency image whose leakage pulls the rectangular-window Goertzel peak low on short buffers (~ 0.3 Hz at 20 cycles), pinning frequency to < 0.02 Hz (`SignalMeasurements.java:198`).

Goertzel single-bin DFT magnitude — `goertzelMagnitude` (`SignalMeasurements.java:314`):
`coeff = 2cos(2 $\pi$ f/fs)`, recurrence `q0 = (x-mean) + coeff.q1 - q2`, magnitude `$\sqrt{(q1^2 + q2^2 - coeff.q1.q2)}$` . DC is subtracted so only the AC component at `f` contributes. $O(N)$ per probe — no FFT needed.

Why raw, not comb output — when a channel's mains comb is on, the worker (`ScopeMeasurementWorker.computeMeasurementOnce`, `ScopeMeasurementWorker.java:392-447`) uses the comb only to make the tone the spectral *peak* (a reliable seed), because the comb's notches sit at every mains harmonic and one can land within a few Hz of the tone (and at high sample rates the comb never settles inside the window) — both pull the combed frequency low. It keeps a raw copy of the selected channel (`rawSelBuf`, `:404`) and re-pins the seed on the raw signal via `refineFrequencyAround` (`SignalMeasurements.java:377`) — a Hann-windowed Goertzel search over the narrow band `seed  $\pm$  FREQ_REFINE_HALF_HZ (2 Hz, :71)`, narrow enough that no mains harmonic ($\geq \sim 50$ Hz away) competes.

2.3 Amplitude / RMS / mean / Vpp measurements

In `SignalMeasurements.compute` (`SignalMeasurements.java:80-96`): a single pass accumulates `sum`, `sumSq`, `min`, `max`. Then:

- **Vmean** = `mean · peakVolts` (DC offset in volts).
- **Vpp** = `(max - min) · peakVolts`.
- **Vrms** = AC RMS — `$\sqrt{(\text{variance})} \cdot \text{peakVolts}$` with `variance =  $E[X^2] - E[X]^2 = \text{sumSq}/n - \text{mean}^2$` (`SignalMeasurements.java:92`). The DC bias is deliberately removed so `Vrms` doesn't degenerate to $\approx$ `Vmean` on a DC-biased noise signal.

Rise/fall time — `riseTime/fallTime` (`SignalMeasurements.java:116-153`): bracket-based 10%↔90% transitions, thresholds `lo = min + 0.1·(max-min)`, `hi = min + 0.9·(max-min)`, averaged over all complete transitions with linear interpolation for sub-sample resolution.

Duty cycle — fraction of samples $\geq$ the half-amplitude threshold, `highCount/n` (`SignalMeasurements.java:108, :214`).

Online window statistics — `MeasurementStats` (`MeasurementStats.java`) accumulates `avg/min/max/ $\sigma$` via **Welford's method** (`mean +=  $\delta/\text{count}$` , `m2 +=  $\delta \cdot (v - \text{mean})$` , `$\sigma = \sqrt{m2/(\text{count}-1)}$`), skipping NaN so missing-cycle ticks don't poison the window. Aggregated over the user's averaging window by walking the history ring (`prepareMeasurementRows`, `ScopeView.java:953`). `MeasurementRow` (`MeasurementRow.java`) picks an SI prefix from the value magnitude so `cur/avg/min/max/ $\sigma$` share a unit; frequency is capped at 7 sig-digits (`fmtFreq`, `:71`).

Dual-tone — two simultaneous fundamentals have no single $T_p/f/duty$, so the worker clears all time fields via `withoutTimes` (`SignalMeasurements.java:353`), rendering ---.

2.4 Lanczos sinc reconstruction (display interpolation + trigger refine)

`ScopeLanczos.lanczos` (`ScopeLanczos.java:82`) reconstructs the trace at any fractional sample position t from a precomputed phase table — a band-limited (Whittaker–Shannon) interpolation. Kernel: windowed sinc $\text{sinc}(x) \cdot \text{sinc}(x/A)$ with **A = 16 lobes** each side (`LANCZOS_A`, ≥ 80 dB stop-band, :45). $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ with an 8th-order Taylor series for $|x| < 0.1$ to avoid cancellation near zero (`ScopeLanczos.sinc`, :187).

The `scale` parameter widens the kernel to act as an **anti-aliasing low-pass at the output rate** (`scale = max(1, samplesPerPx)`), killing energy between the output Nyquist and input Nyquist that would otherwise fold into spurious beat envelopes. Kernels are pre-baked into a 1024-phase table (`buildKernelTable`, :155), each row normalized to unit DC gain ($\sum w = 1$) so narrow-V/div gain errors (multiplied by $\sim 3 \cdot 10^5$ `vScale`) don't shrink the amplitude. A two-slot cache keeps `scale==1` permanently (used by trigger refine and fast-time renders) plus one other scale, and the lookup is split from the synchronized cache-miss path so HotSpot C2 can inline the hot loop (`getKernelTable`, :124). The inner sum subtracts a `baseline` (the centre sample) before weighting so accumulation reflects sample *differences*, not absolute magnitudes — critical at extreme V/div where $\sim 1e-7$ float epsilon $\times \sim 3e5$ `vScale` becomes a visible pixel deflection (`ScopeLanczos.java:95–109`). `ZoomedView` has its own simpler non-cached Lanczos for the 1-second overview strip (`ZoomedView.java:190`).

2.5 DSP filters

Chebyshev Type I low-pass (`LowPassFilter.java`) — an 80 kHz HF spike-removal filter (`LpfMode.HZ_80`), cascade of 2nd-order biquads, order 8 (`SCOPE_HF_LPF_ORDER`, `ScopeMeasurementWorker.java:65`). Type I with 0.5 dB pass-band ripple (`RIPPLE_DB`, :53) chosen over Butterworth for a steeper transition. Each conjugate analog pole pair is mapped via the **bilinear transform with frequency pre-warping** $kWarp = \tan(\pi \cdot fc/fs)$ (`LowPassFilter.java:87`); poles come from $v0 = \text{asinh}(1/\epsilon)/N$, $sp = -\sinh(v0) \cdot \sin(\theta) \cdot kWarp$, $op = \cosh(v0) \cdot \cos(\theta) \cdot kWarp$ (:82–93); sections are normalized to unity DC gain and run in Direct-Form-II transposed (`process`, :122). Below Nyquist gating it is `inactive` and passes through (so 80 kHz is a no-op at 48/96 kHz, only working at high sample rates). Reset before each overlapping read block.

Note: the Javadoc and the `order` parameter doc say "Butterworth" but the implementation is Chebyshev Type I (the class-level and `RIPPLE_DB` comments are correct) — a stale doc comment.

Sliding-window median de-spike (`MedianFilter.java`) — `LpfMode.DESPIKE`, window 7 (`LpfMode.java:28`). Outputs the median of the window centred on each sample (`process`, :52; insertion-sort median, :67), removing impulsive outliers up to `window/2` wide while preserving edges with no ringing. Memoryless across blocks (no reset, no edge transient); window edges shrink rather than wrap.

Time-domain mains comb (`MainsCombFilter.java`) — frequency-tracked IIR feedback comb $H(z) = (1 - z^{-N})/(1 - \alpha \cdot z^{-N})$, $N = \text{sampleRate}/f_0$, placing notches at DC and every harmonic $k \cdot f_0$ (:24). Pole radius $\rho = \exp(-\pi \cdot BW/fs)$, $\alpha = \rho^N$ sets the uniform -3 dB notch width (2 Hz in the scope, `MAINS_NOTCH_BW_HZ`). Fractional N uses linear interpolation between the two straddling integer taps (`process`, :345).

- **Tracking** (`track`, :289): Hann-windows the reference, then scans the 50 Hz and 60 Hz bands **plus their 2nd harmonics (100/120 Hz)** with Goertzel power (`scanBand` + parabolic sub-step refine, :395), auto-detecting 50 vs 60 by summed H1+H2 energy and locking from whichever harmonic is stronger (H2/2 when the 2nd dominates — robust to rectifier hum). Lock requires the line to beat the median scanned baseline by `LOCK_RATIO = 4` (~6 dB); a detection is EWMA-smoothed (`LOCK_SMOOTH = 0.97`) and outliers > 0.5 Hz are rejected (:316 – 337).
- **DC-preserving application** (`processPreservingDc`, :378): the comb inherently notches DC, which would zero the trace's mean; this variant subtracts the block mean, combs, then adds it back so only hum (not DC) is removed — keeping `vmean` meaningful.

The scope re-tracks at most every 200 ms (`MAINS_TRACK_PERIOD_NANOS`), and `reset()`s+re-applies the comb each paint over the contiguous read window so its start transient stays off-screen-left in the pre-roll (`applyMainsSuppression`, `ScopeView.java:1244`). (This is the *time-domain* comb; the FFT module uses a separate frequency-domain `applySpectrumCorrection` divide, also in this class at :203, not used by the scope.) Because the comb's delay lines start zeroed each pass, the worker measures the **settled tail** ($\approx 3 \cdot f_s / (\pi \cdot BW)$ samples in, capped at half the window) rather than the un-suppressed head when computing `Vpp/Vrms` (`ScopeMeasurementWorker.java:431`).

Filter order of application per paint/measurement: HF LPF/despoke → mains suppression → trigger /draw/measure (`ScopeView.java:1877`, `ScopeMeasurementWorker.java:382`).

Adaptive alternatives to the comb. `enums/MainsSuppression.java` offers four per-channel scope modes (`oscLeftMainsSuppression/oscRightMainsSuppression`): `NONE`, `IIR_COMB` (above), `SYNC_SUBTRACT` and `LMS`. All three active filters share the `MainsFrequencyTracker` 50/60 Hz lock (Goertzel dual-band scan + parabolic sub-step + EWMA, the same math the comb's `track()` uses) and all apply through `processPreservingDc` (hum out, operating-point DC kept). Unlike the comb — which `reset()`s each paint so its start transient stays off-screen — the two adaptive filters **retain state across paints** so their learned estimate keeps converging.

- **Synchronous template subtraction** (`MainsSyncSubtractFilter.java`) learns one period-locked template of `TEMPLATE_BINS = 512` phase bins spanning exactly one mains cycle — effective rate $512 \cdot f_0 \approx 25.6$ kHz, so any in-band test tone aliases *above* the audio band and is not captured. Mains phase is tracked in *periods* `[0,1]` by a continuous accumulator advanced by the inter-call sample-index delta (:145), keeping it aligned across the scope's non-contiguous snapshots and the FFT's overlapping windows. At fractional phase the template is linearly interpolated (`bin0 = [φ·M]`, weight `w`), the residual `r = x - est` is output, and the two straddling bins are LMS-nudged `template[b] += MU·r·weight` with a deliberately small `MU = 0.001` (:65) so non-periodic tones average out instead of leaking in (~5 s convergence, fine for rock-stable mains). `processPreservingDc` subtracts only `est - mean(template)` so block DC survives (:156).
- **LMS line canceller** (`MainsLmsFilter.java`) models the mains as fundamental + harmonics up to `MAX_HARMONIC_HZ = 1000` Hz (`kMax = [1000/f0]`), each a **quadrature reference pair** (`cos kφ`, `sin kφ`) with adaptive weights `wc[k]`, `ws[k]`. Per-sample the references advance by a **Rodrigues rotation** $(c, s) \rightarrow (c \cdot c_1 - s \cdot s_1, s \cdot c_1 + c \cdot s_1)$ (:154) to avoid per-sample trig; the window-start phase rides the same inter-call-delta accumulator. Each sample subtracts `est = Σk wc[k]·cos kφ + ws[k]·sin kφ`, outputs `error = x - est`, then updates `wc[k] += MU·error·cos kφ`, `ws[k] += MU·error·sin kφ` with `MU = 5e-4` (:57) setting the notch bandwidth. The model carries no DC term, so DC is preserved by construction.

Only the comb reaches the FFT plot path (as the frequency-domain divide, §3.15); `SYNC_SUBTRACT` and `LMS` are scope-time-domain only.

2.6 Dual-tone beat-envelope reconstruction

`reconstructBeatSignal ( ScopeView.java:2194 )` recovers the slow $\cos((F1-F2)/2 \cdot t)$ modulator of $\sin(F1 \cdot t) + \sin(F2 \cdot t) = 2 \cdot \sin((F1+F2)/2 \cdot t) \cdot \cos((F1-F2)/2 \cdot t)$ so the trigger can lock onto the beat rather than jitter between carrier zeros:

1. **Rectify + cascaded boxcar LP** — `|data|` smoothed by two running-mean boxcars of length $L \approx \text{quarter-beat-period}$ in cascade (sinc^2 response, deeper stopband), each emitting at its window centre for zero net group delay (:2222 – 2244).
2. **Synchronous I/Q detection** — since `|cos|` expands to a Fourier series whose first AC harmonic sits at $(F1-F2)$ with phase $2 \cdot \phi_m$, the I/Q correlation of `absLp - DC` against `cos/sin(2 $\pi$ ·beatHz·i)` gives `phase = atan2(Q,I) = 2· $\phi_m$` (:2271 – 2289).
3. **Output** a clean `rawPeak · cos(omegaMod·i - phaseMod)` (modulator frequency/phase halved), peak-matched to the raw signal's `|F1|+|F2|` peak so the overlay touches the carrier envelope (:2290). The standard trigger then runs on this envelope (sinc refine disabled), and `drawBeatOverlay ( :2141 )` optionally paints it. Generator frequencies are FFT-bin-snapped first (`FftBinSnap.snapIfEnabled`) so the reconstructed `|F1-F2|` matches what is actually emitted (`ScopeView.java:1982`).

2.7 Full-period windowing & zero-crossing file save

`ScopeFileSaver ( ScopeFileSaver.java )` snaps the saved range to **rising zero crossings one period apart** so a looped player has no seam click. `findFullPeriodWindow ( :71 )` computes `periodSamples = round(sampleRate/freqHz)`, finds the first rising zero-cross within the first period (`firstRisingZeroCross, :96, condition arr[i-1] < 0 && arr[i] >= 0`) and the last rising zero-cross within the final period (`lastRisingZeroCross, :104`), and saves `[startSnap, endSnap)` — an integer number of periods, both endpoints at amplitude ≈ 0 rising. Falls back to the trivial `(0, actual)` window when no frequency is detected. The detected frequency comes from `ScopeView.getLastFrequencyHz ( ScopeView.java:633 )`. Samples are quantized to signed PCM at the chosen bit depth in 4096-frame chunks (`save, :118`).

2.8 Volts/div & time/div "nice number" step math (OscParse)

`OscParse ( OscParse.java )` holds the **1-2-5 decade** step lists for V/div (1 $\mu\text{V}/\text{div}$... 500 V/div) and t/div (1 $\mu\text{s}/\text{div}$... 1 s/div) and converts between labels and base units. `parseUnit ( :137 )` splits the numeric and SI-prefix parts ($\mu/u=1\text{e-}6$, $m=1\text{e-}3$, bare=1). `voltsPerDivTargets/timePerDivTargets ( :79, :88 )` expose ascending value arrays for the `StepSelector` to step through. `formatWithUnit ( :112 )` auto-prefixes the mantissa into `[1, 1000)` and strips trailing zeros. `tryParseStepInput ( :160 )` is a lenient parser for editable fields, accepting short forms like 100m, 100mV, 5 (bare $\rightarrow$ base unit), with $n=1\text{e-}9$ through bare=1; returns `null` on non-numeric input. The label arrays are kept manually in sync with `MainWindow.VOLT_PER_DIV/TIME_PER_DIV`.

2.9 Graphics: scaling, polyline rendering, decimation, grid

Value $\rightarrow$ pixel scaling — per channel `vScale = peakVolts/vDiv · pixelsPerDivY` with `pixelsPerDivY = h/DIVISIONS_Y`; the zero-line sits at `centerY = h · offsetFrac` (offset slider), and a pixel `y = centerY - (sample - dcOffset)·vScale` (`renderTraces, ScopeView.java:2357–2368`; `drawTrace, :2436`). Time scaling: `windowSeconds = timePerDiv · DIVISIONS_X`, `displaySamples = round(windowSeconds · sampleRate)` (:1836).

Two-regime waveform rendering (`drawTrace, ScopeView.java:2415`):

- *Dense / zoomed-in* (`samplesPerPx ≤ MAX_LANCZOS_DOWNSAMPLE = 5`): one reconstructed Y per pixel column — sinc (Lanczos, scaled) when enabled, else per-pixel linear interpolation between samples (:2438) — stroked through the shared clipped-Path `paintPolyline` (the same renderer as the FFT/FreqResp traces), with AA on and round caps/joins. The `subSampleOffset` shifts the rendered signal by a fraction of a sample so the trigger lands on the centre pixel.
- *Sparse / slow time-div* (`samplesPerPx > 5`): **per-column min/max bars** — for each pixel column, scan the covered samples and draw a vertical line from min to max (:2486), with AA off and flat caps for speed (state saved/restored so the next channel's curve isn't thinned). `ZoomedView.drawTrace` uses the same two-regime split (`ZoomedView.java:215`).

Sample dots — when sample spacing exceeds 10 px (`pxPerSample > 10`), a filled `dotDiameter` oval is overlaid at each sample, sub-sample-shifted by `subSampleOffset · pxPerSample` (`ScopeView.java:2452`).

Grid / graticule — linear 10×10 division grid via the shared `drawGrid` with a centred crosshair carrying `TICKS_PER_DIV` minor ticks (`paintCanvas`, `ScopeView.java:728`). `ZoomedView` draws its own grid + midline (:126). Sliders are dashed cross-hairs with filled triangle handles (`drawSliders`, :1359); ±FS boundary lines, edge voltage/time labels (`drawEdgeLabels`, :1643), and a measurement table complete the overlay.

Anti-aliasing — curves use `SWT.ON` AA; grid and bars use `SWT.OFF` (sharp). The **CPU/SWT software path has no persistence** — each paint redraws a single frame; a frozen `RenderedFrame` snapshot (windowed samples + render params, :1151) lets the screenshot pane reproduce the exact on-screen (possibly frozen) trace. Digital-phosphor afterglow exists only on the GPU path — see §2.11.

2.10 Capture-threading & buffering model

The view reads a shared `SignalBufferReader` ring relative to its `writePos` via `readEndingAt / readLatest`; no read cursor is mutated, so the live capture and the scope never contend (`ScopeView.java:96`). The measurement worker (`ScopeMeasurementWorker`) runs a daemon thread at a fixed **10 Hz** drift-compensated cadence (`MEAS_COMPUTE_PERIOD_NANOS`, :52; loop :279) so avg/min/max/σ samples are evenly spaced; it caps each read at `MEAS_MAX_SAMPLES = 96000` (:59) and excludes a 500 μs `AC_WARMUP_NANOS` prefix to dodge the ADC startup transient that would bias the published DC mean (:362). Results land in a 1024-deep history ring (`measHistory`, :107) guarded by `measHistoryLock`; the paint thread reads snapshots and walks the ring under that lock (`walkRecentHistory`, :183; `averagedChannelMean`, :200). The AC-coupling DC subtraction uses a ≥ 500 ms-averaged per-channel mean (`AC_DC_MIN_AVG_NANOS`, `ScopeView.java:295`) so the trace and trigger don't wobble between worker ticks. `cap/s` is an EMA of the inter-new-frame interval, decaying toward zero on frozen frames (`updateCaptureRate`, :770).

2.11 Digital-phosphor persistence (GL path)

`ScopePhosphor.java` gives the GPU (GL / NanoVG) scope an oscilloscope-style **digital-phosphor afterglow**; the CPU/SWT render path (§2.9) has none, and the feature is flagged "GPU path only" in `Preferences.java:223`. Two off-screen **RGBA16F** framebuffers are kept: a *scratch* buffer holding just the current frame's trace, and a *phosphor* buffer accumulating the decayed history. The 16-bit-float format is deliberate — repeated multiplicative decay reaches zero cleanly instead of leaving the "permanent ghost" an 8-bit texture's integer quantization would (:397).

- **Exponential decay.** On each genuinely new captured frame (`renderer.isLastFrameNew()`, :203) the phosphor is faded by `fade = 1 - exp(-dt/τ)`, `dt = min(now - lastAccum,`

MAX_DECAY_SECONDS) (:262). The `MAX_DECAY_SECONDS = 0.5 clamp (:63)` makes a resume-after-pause fade gently over several frames instead of wiping in one step. The fade is a multiplicative blend `dst ← dst·exp(-dt/τ)` through NanoVG's `GL_ZERO`, `GL_ONE_MINUS_SRC_ALPHA` composite (:330); the fresh scratch trace is then composited over with premultiplied source-over (:266). τ is the user-selected time constant.

- **Modes** (`enums/PersistenceMode.java`, `oscPersistenceMode`): `OFF` (phosphor buffer never created, `render()` returns false (:147), fixed constants `0.5 / 1 / 2 / 5 / 10 / 15 / 20 s`, `INFINITE` ($\tau < 0$ skips the decay step — accumulate forever, (:265), and `MANUAL` (τ read from `oscPersistenceManualSeconds`, `effectiveSeconds()` (:54)).
- **Frame kinds** (:76) pick decay vs wipe per frame: `REALTIME` decays + stamps only on a new frame; `RESET` wipes and re-stamps on a geometry change (pan / zoom / V-div drag) so the trace doesn't smear under the gesture (:297); `CLEAR` wipes without re-stamping on a trigger / signal-source change (:278); `COMPOSITE` re-shows the frozen afterglow on expose / resize without decaying it (:210).

3. FFT Analyzer

This section documents every DSP, mathematical, spectral, averaging and graphics algorithm in the FFT analyzer module. The module computes a live, coherently-averaged spectrum and derives THD / THD+N / SNR / SINAD, harmonics, IMD, a robust noise floor, and mains suppression. Citations are `file:line`.

3.1 The FFT itself — radix-2 Cooley-Tukey, DIT, in-place, parallel

`fft/Fft.java` is a stateless radix-2 **decimation-in-time** Cooley-Tukey FFT operating in place on separate `double[] re, im` arrays whose length must be a power of two (`Fft.java:30-48`, `forward()` at (:74)).

- **Bit-reversal permutation** (`bitReverse`, (:88) reorders the input by reversing the index bits via the standard incremental carry trick, swapping `re[i]/im[i]` with `re[j]/im[j]` for $i < j$. Kept serial (cheap).
- **Butterfly stages** (`stageGroups`, (:138): for stage length `len = 2, 4, ..., N`, each group `[i, i+len)` runs `len/2` butterflies with twiddle factor $W = e^{-j \cdot 2\pi/\text{len}}$ computed once as `cos(ang)`, `sin(ang)` (`ang = -2π/len`, (:79) and advanced incrementally per butterfly via the complex recurrence `cur ← cur·W` (:155-157). Each butterfly is the canonical $u \pm W^k \cdot v$ pair (:147-154).
- **Parallelism** (`butterfliesParallel`, (:106): within a stage the `n/len` groups touch disjoint slices, so for $N \geq \text{PARALLEL_THRESHOLD} = 65536$ (:51) the groups are partitioned across a daemon thread pool (`THREADS = cores-1`, (:53) with `invokeAll` acting as a per-stage barrier (:127). Below threshold, or on ≤ 2 -core machines, it stays serial. Results are bit-identical to the serial path — only the partitioning differs.
- **Real-input handling**: the analyzer feeds a real signal with `im[]=0` and then uses only the single-sided spectrum bins `0..N/2`; the IFFT for the Dolph-Chebyshev window exploits `IFFT(W)=FFT(W)/N` for real W (see §3.4).

3.2 Windowing and coherent gain

`FftAnalyzer.buildWindow` (`fft/FftAnalyzer.java:1556`) builds the analysis window of length `N`; the table is cached and rebuilt only on `(N, windowType)` change (`getCachedWindow`, (:130) to avoid $\sim 1M$ `cos/sinh` evals at large `N`. Window types (`enums/WindowType.java`):

- **RECT** — all ones.
- **HANN** — $0.5 \cdot (1 - \cos(2\pi n/(N-1)))$.
- **BH4** — minimum 4-term Blackman-Harris (Harris 1978), coefficients 0.35875, 0.48829, 0.14128, 0.01168.
- **BH7** — 7-term Blackman-Harris (Nuttall 1981), alternating-sign cosine sum.
- **FT** — SRS SR785 5-term flat-top, **periodic** N normalization for amplitude accuracy (all others use $N-1$ symmetric).
- **HFT144D / HFT248D** — Heinzel high-flattop windows (144 / 248 dB sidelobes, amplitude-flat; from the GH_FFT report's coefficient catalogue).
- **KB24 / KB38** — Kaiser windows $w[n] = I_0(\beta \cdot \sqrt{1-x^2}) / I_0(\beta)$ at $\beta = 24$ (≈ -226 dB sidelobes, narrower main lobe than equally-suppressing cosine windows) and $\beta = 38$ (below the double-precision FFT floor).
- **DC150 / DC200 / DC250 / DC300** — Dolph-Chebyshev windows at 150–300 dB sidelobe attenuation (§3.4).

Coherent gain is computed as $\text{cohGain} = (\sum w[n]) / N$ (:444-448) and folded into the amplitude normalization $\text{normFactor} = 1/(N \cdot \text{cohGain})$ (:1106), so a windowed tone reads its true amplitude. The single-sided spectrum multiplies non-DC/Nyquist bins by 2 (:1138-1140). `MathUtil.chebyshevT/acosh` (`fft/MathUtil.java:38,49`) supply the Chebyshev polynomial (trig form for $|x| \leq 1$, hyperbolic outside) used by the DC windows.

3.3 Overlap processing

`enums/FftOverlap.java` defines overlap fractions 0 / 50 / 75 / 87.5 / 93.75 %. The hop is $\text{step} = \text{round}(N \cdot (1 - \text{fraction}))$ (`FftAnalyzer.java:423`) and the frame count is $(\text{len} - N) / \text{step} + 1$ (:424). This is **overlap analysis** (successive windowed FFTs sharing samples), not overlap-add resynthesis. The GUI worker slides its retained analysis window forward exactly one hop per tick (`FftAnalyzerWorker.java:1424-1436`), so the overlap region is reused without re-reading the capture ring.

Practical note: high overlap is data-limited — 93.75 % gives 16 frames sharing 93.75 % of their samples (correlated noise), so the "N averages" counter runs $\approx 16\times$ the effective independent averages and the noise floor falls at the per-second *data* rate regardless of overlap. ~ 75 % recovers the window's edge taper; beyond that is mostly extra FFT compute for the same floor-per-second.

3.4 Dolph-Chebyshev window via inverse-DFT

`buildChebyshevWindow` (`FftAnalyzer.java:1631`) builds the equiripple window for a target sidelobe attenuation `attenDb`:

1. $\beta = 10^{(\text{atten}/20)}$, $x_0 = \cosh(\text{acosh}(\beta)/(N-1))$ (:1632).
2. Frequency samples $W[k] = T_{\{N-1\}}(x_0 \cdot \cos(\pi k/N))$ filled symmetrically (:1639).
3. Time-domain window via $\text{IFFT}(W) = \text{FFT}(W)/N$ (real W , :1649).
4. `fftshift` by $N/2$ to centre the main lobe (:1657).
5. Normalize to peak 1 (:1662).

3.5 Sub-bin frequency estimation (kFractional) and harmonic synthesis

The true tone frequency is $k_f \cdot F_s / N$ with non-integer k_f ; using the integer bin for harmonic de-rotation causes $2\pi \cdot \varepsilon \cdot n / N$ per-sample phase drift and total cancellation over long records (:451-457).

- **Phase-difference estimator** (:780-792): two frames `step` samples apart give $k_f = (\Delta\phi - 2\pi m) \cdot N / (2\pi \cdot \text{step})$ after integer-cycle disambiguation $m = \text{round}((\Delta\phi - \text{expected}) / 2\pi)$ — window-independent, accurate to $\sim 1e-5$ bin.
- **Parabolic interpolation** fallback (`MathUtil.parabolicBinInterp`, `MathUtil.java:61`): three-point log-power parabola $\delta = 0.5 \cdot (\alpha - \gamma) / (\alpha - 2\beta + \gamma)$ when only one clean frame exists (:794).
- **Long-baseline κ refit** (`refineKappaOverSegment`, :236): regresses the fundamental's de-rotated phase **slope** over every clean frame using a single-bin **Goertzel** at `refIntBin` ($O(N)$ /frame, $\sim 6\%$ of a full FFT) instead of one hop. Residuals are recentred on their **circular mean** (wrap-safe) and offsets centred for conditioning; the slope yields κ with a $(\text{bestLen}-1) \cdot \text{step}$ baseline (:260-272), shrinking residual drift $\sim$ that factor. Applied only for coherent averaging with ≥ 4 clean frames and sanity-gated to $|\Delta\kappa| < 0.5$ (:863-877).
- **Harmonic detection** (`detectHarmonics`, :1753): physics pins the N th harmonic at fractional bin `N·kFractional`; the two straddling bins are **power-weighted** by the sub-bin offset $w = |\text{offset}| \in [0, 0.5]$: $|X|^2 = (1-w) \cdot |X[\text{main}]|^2 + w \cdot |X[\text{adj}]|^2$ (:1778-1781), recovering energy split by window leakage instead of a local-max search that snaps to noise.
- **Second-tone estimate**: the same clean-frame phase-difference/parabola method runs around a hinted or auto-detected second tone (:802-855), so dual-tone readouts report true sub-bin frequencies, not peaks off the collapsed average.

3.6 Coherent (vector) vs incoherent (power) averaging and per-lobe de-rotation

Two averaging modes (`coherentAveraging` flag, `FftAnalyzer.java:594`):

- **Coherent** — sum complex spectra after de-rotating each frame to a common time origin; noise (random phase) cancels, SNR improves $\sim \sqrt{\text{frames}}$ (:1127-1131).
- **Incoherent** — sum per-bin power $|X[k]|^2$ (no phase alignment), $\text{mag} = \sqrt{\text{sum}/\text{fc}}$ (:1132-1136).

The key coherent algorithm is **per-lobe constant-phase de-rotation** (Pass 2, :953-1042): the fundamental's inter-frame phase advance is $\Phi = -2\pi \cdot k_f \cdot f \cdot \text{step} / N$ (:986); each bin is snapped to its nearest harmonic $h = \text{round}(\text{signed-bin} / k_0)$ and de-rotated by that lobe's **constant** phase $h \cdot \Phi$ (cached via an incremental phasor; negative h is the conjugate, :991-996). Crucially this is **not** a per-bin frequency ramp — a ramp matches only at the exact harmonic bins and over-rotates the leakage skirt $\propto (\text{bin-offset} \times \text{frame})$, producing a Dirichlet/array-factor comb at $F_s / (\text{frames} \cdot \text{step})$ on the skirt. Constant phase per lobe aligns the whole lobe, killing the comb (:960-970).

3.7 Dual-tone / IMD-grid de-rotation

For a genuine second tone, single-reference de-rotation would snap F_2 to a harmonic of F_1 and can cancel it 180° out (the dual-tone IMD bug). The analyzer tries both $h \cdot \Phi(F_1)$ and $\Phi(\kappa F_2)$ de-rotations over the segment and keeps whichever leaves F_2 stronger (:879-911); the wrong one cancels, so the strongest self-selects. It then builds an **IMD product grid** $a \cdot \kappa_{F1} + b \cdot \kappa_{F2}$ ($|a| + |b| \leq \text{order}$, $b \neq 0$) and de-rotates each product lobe by $a \cdot \Phi(F_1) + b \cdot \Phi(F_2)$ over a ± 24 -bin lobe (:912-951, :997-1019), so the whole 2-tone comb de-rotates at its true frequencies. $b=0$ terms stay single-reference.

3.8 Frame rejection (currently disabled) and clean-segment selection

The per-frame R-invariant / phase-coherence rejection is wired but gated off (`frameRejection=false` , :530); when enabled it works as:

- **Pythagorean sine invariant** (:499-523): for $s=A \cdot \sin(\theta)$, $R=\sqrt{(s^2 + (\Delta s/k)^2)} \equiv A$; clean samples give $R \approx A$, glitches deviate. Uses a central difference, operates on the DC-removed signal (`sampleMean/dcRemovedRms` , :1715,1724), with `peakAmp= $\sqrt{2} \cdot \text{RMS}$` . Sample hits are grouped into events ($\text{gap} \leq 16$ samples) and overlapping frames marked dirty (:610-690).
- **Feasibility gates**: skipped when the sine is buried >20 dB below the time-domain peak (:540-543), when the derivative-noise ratio $\text{RMS}(\Delta^2 s)/(A \cdot \omega^2/\sqrt{2}) > 5$ (`derivativeNoiseRatio` , :1969), for multi-tone signals, or when all frames would be rejected (:558-708).
- **Phase-coherence check** (:1044-1101): after correction, frames deviating from the **circular-median** fundamental phase (`circularMedianPhase` , :1948 — median of unit-circle $\sin/\cos$ components) by $>3^\circ$ are re-FFT'd and subtracted from the accumulator.
- **Longest clean run** (:732-743) selects the contiguous clean-frame segment used for the k re-estimate and accumulation.

The live cross-tick safety net against glitchy frames is instead the spectral `SpectralDiscontinuityDetector` (§3.13), with the time-domain Δ^n rejector kept but disabled.

3.9 Magnitude units and reference math

dBFS is the single base scale. `FftResult` carries per-bin amplitudes in dBFS only (no dBV copies; the refined fundamental level is `fundamentalTrueDbFs`). `enums/MagnitudeUnit.java` converts dBFS to the displayed unit at plot time (`convertFromDbFs`):

- **dBFS** — pass-through.
- **dBV** — `dbFs + dbvOffsetDb`, where `dbvOffsetDb =  $20 \cdot \log_{10}(\text{adcFsVoltageRms})$` (cached in `Preferences`, recomputed when the ADC calibration changes) anchors 0 dBFS to absolute voltage.
- **V (Vrms)** — $10^{(\text{dBV}/20)}$.
- **V/ $\sqrt{\text{Hz}}$** — $V / \sqrt{\text{binBw}}$ (PSD, `binBw = Fs/N`).

Because the stored data never carries volts, an ADC recalibration re-labels every displayed value instantly without touching results. The ratio denominator `refLin` for THD/SNR/THD+N/harmonic % is the measured fundamental, or the user-declared manual fundamental (entered in V/dBV, converted once to dBFS) — manual fundamental affects the *reference level only*, never the spectrum. `FftFormat.magToYFraction` maps a value to a vertical fraction, log-scaled for V/V $\sqrt{\text{Hz}}$ axes and linear for dB axes (`gui/fft/FftFormat.java`).

3.10 Noise floor (median + MAD-style), signal masking, THD / THD+N / SNR / SINAD

- **Signal-bin mask** (`buildSignalBinMask` , :1796): fundamental + every harmonic bin, each smeared by $\pm \text{EXCL_BINS}=4$ for window leakage; excluded from both the noise median and SNR sum.
- **Noise floor** (`computeNoiseFloorAndExtendSignalMask` , :1834): two medians in one scan — a **range-restricted median** (spike threshold / SNR) and a full-spectrum **10th-percentile** floor (leakage-immune, closer to true quantization noise than the median which is pulled up by spurs, :1867-1870). A **dynamic fundamental-exclusion walk** widens the mask

outward from the fundamental while bins exceed the 10th-percentile floor (:1876-1908); a **phase-noise zone** within $\pm \text{fundBin}/2$ marks any bin above the refined median as signal (:1910-1924). Sorting uses `Arrays.parallelSort`.

- **THD** = $\sqrt{(\sum \text{harmonic}^2) / \text{refLin}} \times 100\%$ over H2..H9 within the distortion band (:1273-1288).
- **SNR** = $10 \cdot \log_{10}(\text{refLin}^2 / \sum \text{noise-bin power})$ over non-signal bins in-band (:1315-1329).
- **SINAD** = $10 \cdot \log_{10}(\text{refLin}^2 / (\text{noisePower} + \text{harmPowerSum}))$; **THD+N** = **-SINAD**; ENOB derives from SINAD as $(\text{SINAD}-1.76)/6.02$ (:1331-1345). `recomputeStats` (:1418) re-derives all of these from the (possibly post-processed / accumulated) `amplitudeDbFs`, keeping bin positions fixed.

3.11 IMD analysis (CCIF/DIN difference-frequency convention)

`gui/fft/ImdAnalyzer.java` is a pure function over the **dBFS** spectrum (`amplitudeDbFs`; tone /product *ratios* are scale-independent, so no voltage calibration enters the math), producing `ImdResult` (`gui/fft/ImdResult.java`):

- **Tone detection** — `argmax` within ± 8 bins of each commanded frequency, refined by the 3-point quadratic peak interpolation $\delta = 0.5 \cdot (y_L - y_R) / (y_L - 2y_C + y_R)$ with the Smith & Serra interpolated peak value $y_C - 0.25 \cdot (y_L - y_R) \cdot \delta$ (`refinePeak`, :211-237). True sub-bin frequencies come from the analyzer's clean-frame estimates when available (:101-104).
- **Products** (:126-152): $d2L = f2 - f1$, $d2H = f1 + f2$; for order $n \geq 3$, $dnL = (n-1)f1 - (n-2)f2$ (below F1) and $dnH = (n-1)f2 - (n-2)f1$ (above F2), orders 2..5. Plus **DFD2** = $f2 - f1$ and **DFD3** = $\sqrt{(|F(2f1-f2)|^2 + |F(2f2-f1)|^2)}$ (:118-124).
- **Ratios** — reference is the **linear-magnitude sum** $|F1| + |F2|$ (`Vrms`); each product expressed as % of it; `imdPwrPct` is the RMS sum of all products over the same denominator (:113-155).
- **TD+N** (:157-202) — Parseval/window-independent: $100 \cdot (V_{rms} - \sqrt{F1^2 + F2^2}) / V_{rms}$ where V_{rms}^2 sums squared bin voltages over the whole spectrum and the F1/F2 term sums over each tone's **dynamic skirt** (walk from peak to the 10th-percentile floor, `noiseFloorDbV / skirtEdge`, :250-281). The window ENBW cancels in the ratio, so it's correct for any window.

3.12 ToneLobeLift — log-domain lobe stretch (pin feet to noise, not uniform lift)

`dsp/ToneLobeLift.java` rescales a tone's main lobe (for `.frc` calibration $1/|H|$ or manual-fundamental level) as a **vertical stretch anchored on the noise floor**, producing a smooth dome whose feet stay on the grass — explicitly **not** a uniform lift (which would make a flat-topped box with vertical cliffs, :24-54):

1. **Robust floor ceiling** (`localFloor`, :80): `median + 4 · MAD` over wide flank bands (`floorGap / floorSpan`), so a wide lobe / harmonics / mains / spurs falling inside are ignored as outliers.
2. **Lobe extent** (`lobeBins / edge`, :103,108): walk out from the peak until a bin drops to the floor; the peak bin is always included.
3. **Stretch** (`stretch`, :127): $|X'| = |X| \cdot \text{factor}^t$ with $t = \ln(|X| / \text{floor}) / \ln(\text{peak} / \text{floor})$ ($t=1$ at peak, 0 at floor). t is capped at 1 so a noise bin standing above the assumed peak isn't lifted more than the peak (:133-136). Bins at/below the floor are untouched; a sub-floor peak is simply scaled. `FftView.stretchLobe` (`gui/fft/FftView.java:1626`) applies this per tone at plot time.

3.13 SpectralDiscontinuityDetector — 3-gate running-median glitch rejector

`dsp/SpectralDiscontinuityDetector.java` rejects glitchy/stalled blocks **before** they enter the cross-tick vector average, comparing each block's spectrum to the running statistics of accepted blocks (a real line self-references; a glitch lifts the floor or near-carrier pedestal). Every comparison is a self-calibrated dB ratio (`median + k·MAD`, `mad` scaled $\times 1.4826 \approx \sigma$, :420-427). The first block seeds the reference and is accepted unconditionally (:205-213). Bands are **log-spaced** (`buildBands`, :344), narrowed to $\pm \text{FLOOR_GUARD_HZ}$ around each fundamental.

- **Gate 1 — near-carrier pedestal** (:215-223): median power of the skirt flanking each fundamental (the data-derived main lobe excluded via `ToneLobeLift`, `pedestalDb` :275) minus the local floor; the excess is amplitude/leakage-independent, gated against its running `median + 10·MAD` (larger sigma for the heavier leakage tail).
- **Gate 2 — broadband floor lift** (:225-240): per-band running-median reference; mean lift over non-line bands `> median + 6·MAD` rejects a uniform broadband rise the pedestal can't see.
- **Gate 3 — total power** (:242-246): `|powerDb - median| > 6·MAD` catches a generator stall / long dropout where lines collapse instead of the floor rising.

MAD floors (`MIN_*_MAD=0.5`, :64-66) stop a very-stable run from collapsing a threshold. The `Gates` snapshot feeds the debug overlay (:441). In the worker, rejection triggers a re-sync (`onSignalDiscontinuity`, `FftAnalyzerWorker.java:526`).

3.14 Cross-tick coherent accumulation, phase-lock loops and FLL

The worker (`gui/fft/FftAnalyzerWorker.java`) runs a deeper cross-tick average than one `analyze()` call. Each tick's spectrum is rotated to the pinned reference time origin and frame-weighted; forever mode is a true cumulative mean ($\text{SNR} \propto \sqrt{\text{total-frames}}$), a finite ring `N` an exponential window $\alpha = \text{weight}/N$ (`accumulateIntoForeverBuffer`, :595).

- **Tone routing.** `detectStrongTones` keeps a peak as a separate tone only if within `fftStrongToneRelDb` (Preferences ► FFT, default 100 dB) of the *strongest* peak and above a ~ 10 Hz DC-reject floor. 0–1 tones $\Rightarrow$ single-reference path (tighter one-shot κ -refine, harmonics ride the one ramp); ≥ 2 tones $\Rightarrow$ multi-tone path. A clean tone with far-below harmonics should stay single-reference.
- **Single-reference path** (:686-816): pins `round(kFractional)`, does the **one-shot κ refine** from the de-rotated fundamental's phase slope $d\phi/d\Delta = 2\pi \cdot \Delta\kappa/N$ over `KAPPA_MEAS_FRAMES` ≈ 24 clean frames ($\sim 100\times$ tighter than the single-frame peak, :706-740), then runs a continuous **phase-tracking PLL** measuring the running mis-alignment vs the deep accumulated phase (the $\sqrt{N}$ -averaged reference) and folding `PHASE_TRACK_GAIN` of it into `accumDroppedSamples` (:741-773). This continuously re-locks, killing the slow κ -residual drift that a frozen κ would ramp into the de-rotated phase ($2\pi \cdot \Delta\kappa \cdot \Delta/N$, harmonics $h\times$ faster). A re-sync (overrun/discontinuity) is just a big residual $\rightarrow$ one-shot full realign (gain 1) then the loop cleans up. Per-lobe constant-phase de-rotation over the half-spectrum follows (:774-816). A dropout guard holds the accumulator when the fundamental falls below `DROPOUT_FRACTION` of the average (:661-669).
- **Multi-tone path** (`accumulateMultiTone`, :853): each tone (detected from the spectrum, `detectStrongTones` :1041, with parabolic refinement and harmonic rejection) de-rotated by its own constant phase, each carrying its own per-tone PLL; a per-tone residual `> PHASE_JUMP_RAD` $\approx 60^\circ$ triggers a one-shot realign. A pooled clock-drift δ de-rotates non-tone bins, and the IMD grid `a· $\kappa$ 1+b· $\kappa$ 2` rides the tracked tone phases (:943-1005). Its own κ -refine folds each tone's $\Delta\kappa$ once (:1006-1031).

- **overlayAccumulatorOnto** (:1141) rebuilds the displayed dB spectrum from the raw cumulative average, deriving the mag→amplitude factor from the live result so the conversion applies once.
- **DerotationPhaseLock** (fft/DerotationPhaseLock.java) is the standalone, unit-tested **measure-then-correct** lock: holds κ fixed over a window, measures the de-rotated phase slope $2\pi \cdot (\kappa_{\text{true}} - \kappa) / N$, applies one full correction $\kappa += \text{slope} \cdot N / 2\pi$ (:103-134), and declares lock after enough sub- `lockDriftRad` windows (or a `maxWindows` fallback). Unconditionally stable where a naive per-tick PLL diverges.
- **FrequencyFll / FLL** (gui/fft/FrequencyFll.java) steers the **generator** onto the bin grid with one-step deadbeat `correction -= (detected - target)` plus **exact transport bookkeeping**: each issued correction records the absolute capture position from which a measurement can reflect it (`publish-time writePos + a 0.7 s DAC-drain guard`), and every update whose analysis window *starts* earlier is held unconditionally — such a measurement provably predates the correction, so acting on it would stack a second correction onto an error already in flight. This replaces the earlier heuristic transport models (update counts, arrival fractions, measured dead times), which could mis-measure under fine-track noise and re-correct faster than the physical $\sim 3\text{--}4$ s loop delay. Within the steady state it holds inside the `LOCK_PPM = 0.01` ppm band (the measurement's own jitter floor), and a **drift-rate feedforward** (an EWMA-tracked Hz/s slope of the residual error — the relative wander of the two free-running converter crystals, sanity-capped) is applied predictively on every update so the error stays near zero instead of sawtoothing to the band edge once per transport round-trip. `FrequencyAligner / FrequencyAlignerFactory` wire it as the sole `AlignGenerator.FLL` implementation.

The whole per-tick single-reference pipeline is parallelized (chunked disjoint-write phasor de-rotation + `Arrays.parallelSort`), keeping a large-FFT tick under the 93.75 %-overlap hop budget. A coalescing single-slot UI handoff (`AtomicReference<FftResult>`) bounds the repaint backlog to one spectrum so a fast worker can't pile spectra into SWT's `asyncExec` queue and climb to OOM. `.frc` calibration and mains correction are applied at **plot time** on the raw accumulator (a per-bin linear factor commutes with de-rotate-and-sum), so toggling a calibration never corrupts the running average. Deep averaging runs with frequency *align OFF* (the align loop keeps moving the captured frequency, which the de-rotation can't track).

3.15 Frequency-domain mains suppression (plot-time comb correction)

`enums/MainsSuppression.java` selects `NONE` or `IIR_COMB`. The comb is **not** applied to time-domain samples (an input IIR comb convolves its transient/phase into every frame, which a coherent average can't undo); instead the worker only **tracks** the mains f_0 — every 5th analysis tick (`MAINS_TRACK_TICK_INTERVAL` ; the lock is an EWMA with ≈ 33 -detection memory, so the cadence change is negligible while the ~ 208 Goertzel sweeps stay off most ticks' budget) — and the comb's normalized frequency response is divided out of the displayed averaged spectrum at plot time (`dsp/MainsCombFilter.java`). `magnitudeAt` (:174) is the closed form of $H(z) = (1 - z^{-N}) / (1 - \alpha \cdot z^{-N})$ on the unit circle ($\omega N = 2\pi \cdot f / f_0$), peak-normalized by $(1 + \alpha) / 2$ so the passband isn't biased (≈ 1 in passband, $\rightarrow 0$ at every $k \cdot f_0$). `applySpectrumCorrection` (:203) adds the cached, band-limited (`CORR_MAX_HZ`), floored per-bin dB correction in place, rebuilt only when f_0 moves. The accumulator stays raw — analogous to the `.frc` cal, which is likewise applied at plot time.

3.16 Shared frequency-domain plotting (AbstractMeasurementView)

`gui/common/AbstractMeasurementView.java` is the shared chart engine reused by both the FFT and Frequency-Response views; `gui/common/AbstractFreqDomainView.java` adds the log-frequency specifics.

"Nice" axis tick generation (`drawGrid`, :539):

- **LINEAR** — `divisions+1` evenly spaced ticks (`linearTicks`, :862).
- **LINEAR_NICE** — majors at $1/2/2.5/5 \times 10^n$ (`niceLinearMajors`, :947: pick the step from the mantissa of `range/targetCount`), with caller-specified minor step (`niceLinearMinors`, :967).
- **LOG** — decade majors 10^n (`logMajorTicks`, :871) with $2..9 \times 10^n$ minors (`logMinorTicks`, :891); labels adaptively thinned by visible decade count (`adaptiveLogLabels`, :1122). A sub-decade zoom (`isSubDecade`, :810) falls back to nice-linear ticks/minors (`subDecadeMinors`, :827). Label placement is pixel-aware and two-pass — decade values reserved first so a round decade is never thinned (:704-716) — with clash skipping by text extent.

Value→pixel mapping:

- `axisFraction` (:930) returns `[0,1]` position — log uses base-10 $(\log_{10}(v) - \log_{10}(\min)) / (\log_{10}(\max) - \log_{10}(\min))$ matching `freqToX`; `valueToX`/`valueToY` map to absolute pixels (Y inverted, :915,922).
- `AbstractFreqDomainView.freqToX/xToFreq` (:164,184) are the `freq↔X` transforms with `safeMin=max(1,freqMin)` and a tiny multiplicative `safeMax` guard; `dbToY` (:227) linear-dB; `magToY`/`magToYTrace` (:236,249) route through `FftFormat.magToYFraction` (clamped for markers, unclamped for the trace so off-range slopes stay correct and the GC clip cuts them flush).

Spectrum polyline rendering (`ColumnBucketPainter`, :1188 via `paintPolyline`, :1162): buckets up to thousands of bins into $\leq \text{plot.width}$ pixel columns. Pass 1 draws one **vertical envelope bar** per multi-point column (AA off, Y clamped to plot so GDI's $\pm 32k$ limit isn't exceeded, :1247-1259); pass 2 strokes a single anti-aliased midpoint `Path` through column centres, each segment **Liang-Barsky-clipped** to the plot (`clipSegmentToPath`, :1301) so far-off coordinates never wrap, with left/right edge anchors so the trace reaches the plot edges. `SKIP_Y` drops NaN samples.

Other shared facilities: `paintCachedStatic` (`AbstractFreqDomainView.java:106`) caches the static layer (grid+axes+traces) keyed by a fingerprint long so crosshair/blink redraws don't re-walk data; `drawReadoutBox` (:276) is the auto-flipping crosshair readout; `drawOutlinedText`/`drawOutlinedDashedLine` (:210,256) give peak labels and markers a background-coloured halo over the live trace.

4. Frequency Response

The Frequency Response module measures a device-under-test's transfer function $H(f)$ — amplitude (dB) and phase (degrees) versus frequency — across the audio band. Unlike a classic stepped-sine sweep, this module uses a **single continuous Farina exponential log-sweep** played once, captured on both ADC channels, and converted to $H(f)$ by **frequency-domain deconvolution** ($H = Y/X$). Both stereo channels are deconvolved in parallel from one playback.

Core pipeline files: `FreqRespAnalyzer.java` (orchestration), `FreqRespCalHelper.java` (the DSP), `SignalGenerator.java` (sweep generation), `FreqRespView.java` (plotting + calibration/RIAA at draw time).

4.1 Farina exponential log-sweep generation

Algorithm. A unit-amplitude exponential sine sweep (chirp), $x(t) = \sin(K \cdot (e^{t/L} - 1))$ with $L = T/\ln(f_1/f_0)$ and $K = 2\pi \cdot f_0 \cdot L$. The instantaneous frequency advances *log-linearly* from f_0 at $t=0$ to f_1 at $t=T$; phase starts at exactly 0 so the sweep concatenates onto leading silence with no discontinuity. Rendered once into a `float[]` buffer (`SignalGenerator.renderLogSweep`, `SignalGenerator.java:367-377`; construction at `SignalGenerator.java:336-354`).

Why exponential (Farina) and not stepped-sine. A single continuous sweep excites every frequency in one pass, giving far better measurement-time/SNR than dwelling on N discrete tones, and the exponential profile spends equal energy per octave — matching the log frequency display and human hearing. The *same* `renderLogSweep` output is reused by the analyzer as the deconvolution reference $X(t)$, so X is byte-identical to what the DAC emitted (`FreqRespAnalyzer.java:126`, `getLogSweepBuffer`).

Playback envelope. The sweep is played one-shot (no looping), emitting silence after the buffer ends so a second cycle can't alias into the capture window and pollute the deconvolution (`FreqRespAnalyzer.java:96-106`, `logSweepNext` at `SignalGenerator.java:600`). A **Hann fade-in/fade-out** (a Tukey window) of `SWEEP_FADE_FRACTION_PER_SIDE = 5%` per side is applied to the sweep boundaries to suppress the $1/T$ spectral-leakage ripple (sinc sidelobes) that the abrupt sweep start/stop would otherwise inject into $H(f)$; $5\% \approx 20$ dB suppression (`sweepEnvelope` at `SignalGenerator.java:625`, fade length `FreqRespCalHelper.sweepFadeSamples` at `:106`).

4.2 Capture timing and grids

- **Capture length** = lead-in silence + sweep + a fixed $\frac{1}{2}$ -second tail (so the device's impulse-response decay is fully captured before the buffer ends), rounded up to whole seconds (`FreqRespAnalyzer.java:78-82`). Lead-in lets the DAC/ADC chain settle before the first sweep sample.
- **Output frequency grid.** N geometrically (log) spaced points from `startHz` to `stopHz` (`buildLogSpacedFreqs`, `FreqRespAnalyzer.java:183`): $f[i] = \exp(\logStart + (\logEnd - \logStart) \cdot i / (N-1))$. A compile-time switch `USE_LOG_GRID` (`:61`) can instead emit linear-spaced points (`buildLinearFreqs`, `:197`) for consumers wanting integer-Hz bins; production uses log. The CLI (`FreqRespMode.java:137`) builds the identical log grid.
- **Single-pass stereo.** One playback, both ADC channels kept; L and R are deconvolved on separate `CompletableFutures` sharing the reference sweep (`FreqRespAnalyzer.java:127-138`; CLI `FreqRespMode.java:203`).

4.3 Frequency-domain deconvolution $H = Y/X$

This is the heart of the module (`FreqRespCalHelper.computeFromLogSweep`, `FreqRespCalHelper.java:136-370`).

1. Build matched X and Y buffers. The reference X is rebuilt with the *same* Hann fade the player applied (`:153-164`) — so Y and X carry identical boundary shapes and the start/stop leakage cancels in the division. Both are zero-padded to $M = \text{nextPow2}(\max(yLen, \text{leadIn} + \text{sweepLen}))$ (`:144`). The captured Y is placed at sample 0; the reference X is offset by `leadInSamples` to match.

2. Real FFT both, complex divide per bin. Using `JTransforms.DoubleFFT_1D.realForward`. For each half-spectrum bin k : $H = Y/X$ computed as $H = (Y \cdot \text{conj}(X)) / |X|^2$ — $hRe = (y_r \cdot x_r + y_i \cdot x_i) / |X|^2$, $hIm = (y_i \cdot x_r - y_r \cdot x_i) / |X|^2$ (`:177-191`). Bins where $|X|^2=0$ (outside the swept band) are set to 0.

Why deconvolution over per-bin readout. Dividing the full captured spectrum by the full reference spectrum recovers the *entire* transfer function in one FFT pair, with the sweep's energy-per-octave weighting giving good SNR at every frequency — superior to reading single FFT bins of a stepped tone, and it naturally yields phase as well as magnitude.

3. Transport-delay removal (DAC↔ADC clock/latency). $H(f)$ is inverse-FFT'd to the impulse response (IR). The IR's main peak gives the round-trip transport delay; sub-sample delay is refined by a **parabolic (quadratic) interpolation** of the three samples around the peak: $\delta = \text{peak} + 0.5 \cdot (y_{-} - y_{+}) / (y_{-} - 2y_0 + y_{+})$ (:201-216). This delay is then removed as a pure **linear-phase rotation** applied bin-by-bin in the frequency domain, $H(k) \cdot= e^{+j \cdot 2\pi k \cdot \delta / M}$ (:218-226). This is how the module handles the DAC-playback vs ADC-capture timing offset — and it keeps notches' physical $\pm 180^\circ$ phase steps intact instead of smearing them (rationale in `FreqRespMode.java:67-72`). Note: `ClockMismatch.java` (which maps a measured-vs-generated frequency error in ppm back to the 22.5792/24.576 MHz master oscillators) is used by the *stepped-sine* iterative/FFT modes, **not** by this deconvolution path — here the clock/latency offset is absorbed entirely by the IR-peak delay term.

4. Optional IR time-gating (disabled by default). `USE_IR_GATING=false` (:67). When on, after delay correction the IR sits at sample 0; a symmetric Hann-tapered window of half-width `IR_GATE_LENGTH_SEC` (0.25 s) keeps the main response and zeroes the noise tail, then re-FFTs — trading frequency resolution ($\approx 1/\text{gate}$) for ~ 20 dB less residual ripple (:236-278).

5. Sample H onto the output grid. For each requested frequency, **linear interpolation in the complex spectrum** between the two straddling FFT bins (:284-296): `bin = f/binHz`, `blend hRe/hIm`, then `magLin = hypot(re,im)`, `phaseRad = atan2(im,re)`.

6. Absolute normalisation. Magnitudes are divided by `amplitudeVrms / adcFsVoltageRms` (the captured ADC peak in normalised units for a unity loopback), so a calibrated unity-gain loopback reads exactly 1.0 (0 dB) (:298-316). The comment notes the older `dacDrivePeak` form introduced an offset when DAC FS $\neq$ ADC FS (e.g. +0.84 dB) — using the ADC full-scale RMS fixes it.

4.4 Savitzky-Golay smoothing

Algorithm. The final per-output-point magnitude and phase arrays are smoothed with a Savitzky-Golay filter (`USE_SAVGOL_FILTER=true`, window 7, order 3; `FreqRespCalHelper.java:87-98`, applied : 322-344). SG fits a low-order polynomial over a sliding window via least squares and takes the central tap — it suppresses noise ripple while preserving peaks/notches far better than a moving average. Coefficients are the first row of $(V^T V)^{-1} V^T$ (Vandermonde $V[i][j] = (i - \text{centre})^j$), inverted by plain **Gauss-Jordan** (`savGolCoefficients` :379, `invertSquareMatrix` :429). Magnitude is smoothed directly; **phase is smoothed in (sin, cos) space and recombined via atan2** so $\pm\pi$ wraps at notches don't corrupt it (:327-341). Boundaries use reflection (`applySavGol` :410). A 7-point window over ~ 200 log points $\approx 1/12$ -octave smoothing.

4.5 The .frc correction store (calibration)

Purpose. Cancel the measurement bench's own response (soundcard, cabling, the loopback DAC→ADC path) so only the DUT remains. A calibration is itself a stereo frequency-response measurement saved to a `.frc/CSV` file.

File format (`FreqRespCalHelper.saveCsv` :493, `loadCsv` :538): `#`-comment header (kind, `format_version=6`, `sample_rate`, sweep range/points, DAC drive V RMS) then rows `frequency_hz`, `mag_left_dB`, `mag_right_dB`, `phase_left_deg`, `phase_right_deg`. Magnitudes are stored as **relative**

dB (ADC/DAC voltage ratio), so a flat passband sits at 0 dB independent of absolute scaling. On load, dB→linear ($10^{\{dB/20\}}$) and deg→rad. Comma-separated; legacy semicolon files still parse.

In-memory model (`FreqRespCalibration.java`): parallel `freqs/magLin/phaseRad` arrays, ascending freq, passband ≈ 1.0 . Stereo pair = `StereoFreqRespCalibration`.

Interpolation of a stored curve (`FreqRespCalHelper.interpolate` :774). Binary-search the bracketing points, then interpolate with **log-frequency as the independent variable**: magnitude is interpolated in **dB** ($db = db0 + (db1 - db0) \cdot t$, then back to linear), and **phase via complex unit-phasor interpolation** (blend `cos/sin`, recombine with `atan2`) so $\pm 180^\circ$ wraps at notches interpolate correctly. This is the resampler used throughout the freq-resp path — there is **no Lanczos resampling here** (Lanczos lives only in the scope view's time-domain `ScopeLanczos`).

Composition / store ownership (`FreqRespCorrectionStore.java`): an ordered list of loaded `Entry` calibrations plus a transient wizard `direct` (page-1 loopback) buffer. Multiple loaded files **compose by chaining divides** in order at draw time. The FFT pane and FreqResp pane each own a separate instance (plain object, not a singleton). Mutations fire a constructor-injected `changeNotifier` (GUI publishes a bus event; CLI passes null).

Application timing — at plot time, not at measure time. The analyzer returns the **raw** deconvolution; calibration is divided out only when the curve is rendered (`FreqRespAnalyzer.java`: 140-144). This means swapping calibrations re-traces the existing measurement without re-sweeping. In `FreqRespView.applyCurrentCalibration` (`FreqRespView.java`:468), the raw result is cloned, then each store entry (and the wizard `direct`) is divided in via `divideByStereoCal` (:585) — **linear-magnitude divide, phase subtract**, with the cal log-interpolated onto the result grid. Files loaded from disk arrive pre-corrected (`isCalibrationApplied`), so they're not divided again.

4.6 Mains-notch (frequency-domain interpolation)

Optional removal of mains hum and harmonics from the displayed curve (`FreqRespView.applyMainsNotches` :540). For each harmonic of 50/60 Hz up to Nyquist, every point inside $\pm halfWidthHz$ is **replaced by a linear interpolation (in frequency) of the magnitude and phase from the two points just outside the band** (`interpolateAcrossBand` :555). The half-width scales with the deconvolution FFT length, $halfWidth = 1.2 \cdot sampleRate / fftSize$ (:509), so it always spans a fixed number of bin-widths ($\approx \pm 8$ Hz at 48 kHz / 1.1 s). This is a plot-time interpolation across the contaminated bins, not a time-domain comb filter.

4.7 RIAA phono EQ curve

`RiaaCurve.java` evaluates the phono equalisation curve overlaid on / subtracted from the measurement (for testing phono preamps).

Math. The record (encode) transfer function in the s-domain is $H_{record}(s) = (s \cdot T1 + 1)(s \cdot T3 + 1) / (s \cdot T2 + 1)$ with the standard time constants **T1 = 3180 μ s** (50.05 Hz), **T2 = 318 μ s** (500.5 Hz), **T3 = 75 μ s** (2122 Hz), plus optional IEC **T4 = 7950 μ s** (20.02 Hz subsonic high-pass). Magnitude-squared is evaluated directly (`recordDb` :92): $|H|^2 = (1 + \omega^2 T1^2)(1 + \omega^2 T3^2) / (1 + \omega^2 T2^2)$. The **playback (decode)** curve — the canonical "RIAA curve" dropping from +20 dB @ 20 Hz to -20 dB @ 20 kHz — is the negated record curve (`reverse` flag). Everything is **normalised to 0 dB at 1 kHz** (`REF_HZ` , :64) by subtracting `recordDb(1 kHz)`.

IEC amendment. When enabled the record magnitude is **divided** (not multiplied) by the single-pole high-pass $|HP|^2 = \omega^2 T^4 / (1 + \omega^2 T^4)$ (:98-101) — mathematically the inverse of how IEC enters the playback chain, keeping record⁻¹·playback = identity so a flat loopback still reads 0 dB everywhere even with IEC on.

4.8 RIAA overlay, compare/diff trace, anchoring

- **Overlay** (`FreqRespView.drawRiaaOverlay` :885). The analytic RIAA curve is sampled every 5 px across the plot and drawn as a dashed reference. It's **anchored at 1 kHz to the measured trace's 1 kHz value** (`riaaAnchorDb` :907) so the reference lines up with the data.
- **Compare (subtraction) mode** (`getCompareDiff` :976 , `drawCompareTrace` :933). Computes per-point (`measuredDb - RiaaCurve.evalDb`), i.e. the measurement minus the target RIAA curve, so a perfect phono preamp reads a flat 0 dB line. The diff is then **smoothed by a sliding mean in log-frequency space over a 1/W-octave window** ($W = \text{pref, half-width } \log_{10}(2)/(2W)$, two-pointer O(n) running sum, :1013-1041) — chosen over an index-based window because on a 192k-point log sweep a fixed index count covers wildly different bandwidths at low vs high frequency. The smoothed curve is **anchored to 0 dB at 1 kHz** by subtracting its interpolated 1 kHz value (`valueAt1kHz` :1047 / :1380). Min/max over 20 Hz-25 kHz feed the on-screen table. One cached `CompareDiff` object feeds the drawn trace, the min/max table, and the crosshair Δ readout so all three agree by construction.
- **Auto-setup** (`autoSetupCompare` :1086 , `autoSetupMagnitudeRange` :660). Fits the view to the data: vertical window = data min/max + 10% pad (compare mode keeps +20 dB headroom above the peak, ≥ 2 dB minimum span, centred on the signal midpoint when the natural fit is tighter).

4.9 Live metering during sweep

`FreqRespLiveMeter.java` is a compact "input level vs time" chart shown in the busy shell while the sweep runs (REW-style). Per capture block it receives elapsed time + linear RMS; converts to `dbFs = 20·log10(rms)` clamped to a $-100..0$ dB axis. Incoming RMS is first passed through an **exponential moving average** (`EMA_ALPHA = 0.95` , :88 / :147) — at low frequencies each ~1024-sample block holds only a partial cycle so raw RMS swings wildly into a comb of teeth; the EMA collapses that into a steady amplitude envelope. Drawn as a polyline; over `MAX_POINTS` it decimates by 2 to stay responsive on long captures.

4.10 Plotting (freq-resp-specific parts only)

The view extends the shared `AbstractFreqDomainView/AbstractMeasurementView` engine (§3.16), which provides the **log-frequency x-axis**, the **"nice tick" dB y-axis** (`AxisSpec.linearNice` rounds to $1/2/2.5/5 \times 10^n$ steps), the `freqToX(log)/dbToY` transforms, `paintPolyline` with clipping, and the static-layer paint cache. Freq-resp-specific:

- **Axis setup** (`FreqRespView.onPaint` :805-814): `AxisSpec.log(freqMin,freqMax)` for x; `linearNice(magBot,magTop,...)` in dB for the left y; and an optional **fixed $\pm 180^\circ$ phase right-axis** (`AxisSpec.linear(-180,180,8)`) shown only when the phase trace is enabled.
- **Phase trace** plotted as a dotted polyline, `phaseToY` mapping degrees onto the fixed $\pm 180^\circ$ axis (`phaseToYFraction` clamps then maps, `FreqRespFormat.java:132`).

- **Coordinate/format math** (`FreqRespFormat.java`): `freqToXFraction/xFractionToFreq` (log10), `linToDb` (returns -300 for ≤ 0 to avoid NaN), significant-digit dB/phase readout formatters.
- **Crosshair readout** (`drawCrosshair` :1287) interpolates the active trace (log-freq, linear-dB) at the cursor — and in compare mode interpolates the *same* cached smoothed-diff array so the Δ readout matches the drawn curve. Cursor frequency is clipped at `freqRespNyquistFraction × sampleRate`.
- **Zoom/pan**: log-frequency zoom around the cursor (:1485 , geometric), linear-dB magnitude zoom/pan, all persisted to `Preferences` and published on `FREQRESP_RANGE_CHANGED`.

The CLI path (`FreqRespMode.java`) renders an equivalent JFreeChart PNG with a custom `LogAxis` whose `refreshTicks` places majors at $\{1,2,3,5,7\} \times 10^n$, magnitude on the left axis and unwrapped phase (dashed) on a right axis.

4.11 Tune-notch live measurement (looping circular-transform deconvolution)

The Tune-notch wizard (`TuneNotchWizardDialog` + `NotchSweepEngine.java`) shows a passive twin-T notch's magnitude response refreshing a few times a second while the user turns the trimmer, so the null can be walked onto the target frequency by ear/eye. It reuses the freq-resp deconvolution kernel (§4.3) but drives it very differently.

- **Looping capture, no per-sweep re-open.** The engine opens the device once and streams a **continuously looping** Farina log-sweep into a circular ring buffer (`NotchSweepEngine.java:38`), avoiding the ~ 1.5 s open/close cost the main freq-resp pays per measurement. Ring capacity is `max(sampleRate, 4 × sweepSamples)` (:148); the wizard polls `latestPeriod(n)  $\sim 10 \times /s$` to grab the most-recent `n` samples, unwrapping wraparound modulo `ringCapacity` (:191).
- **Power-of-two period $\Rightarrow$ circular transform.** The sweep period is `round(0.26 s × fs)` rounded to the **nearest power of two** (≈ 0.19 – 0.34 s/pass; grab cadence `loopPeriodMs = round(sweepSamples/fs × 1000)`, a few passes a second) (`TuneNotchWizardDialog.java:501`, :507). With the period a power of two and lead-in zero, `computeFromLogSweep`'s `M = nextPow2(...)` equals `sweepSamples` exactly, so the FFT is **circular, not zero-padded**. A window grabbed at an arbitrary ring phase is then a circular *shift* of the periodic signal — and a shift preserves $|H(f)|$, so **no trigger or loop-cycle alignment is needed** and there is no wrap-smearing. (A non-power-of-two period would zero-pad, an unaligned grab would wrap across the seam, and every bin would spike.)
- **Response per pass.** Each grab deconvolves L and R separately via `computeFromLogSweep` (:597) — $H(k) = Y(k)/X(k)$, magnitude $|H|$ on a **bin-aligned** output grid (:531, exact FFT-bin centres, no faceting), normalized by ADC peak. Loop-seam Hann fades (`FADE_SEC = 0.025 s`, :504) on both the played sweep and the deconvolution reference cancel in Y/X .
- **Savitzky-Golay is OFF here** (`applySavGol = false`, :600). The bin-aligned grid is coarse (a few Hz/point over the 600-px chart); the fixed 7-point SG window would span ~ 15 – 20 Hz and round ~ 9 dB off the bottom of a sharp null. The main freq-resp pane uses a denser log grid where the same window is sub-Hz, so it leaves SG on.
- **Null tracking.** Per result the deepest notch is found by `argmin` of the linear magnitude, then a 3-point **parabolic interpolation in linear magnitude** ($\delta = \frac{1}{2}(y_{-1} - y_{+1}) / (y_{-1} - 2y_0 + y_{+1})$, :651) refines the null frequency and depth to sub-bin (linear, not dB — the dip is locally parabolic in linear space); (`notchHz`, `notchDb`) drive the on-chart readout (:634).

5. ADC Characterisation

Beyond the live GUI, the CLI (`--mode ...`) characterises an ADC's linearity from a captured tone. These paths live outside the four GUI modules and are documented here for completeness; the math is code-density (histogram) and least-squares regression.

5.1 Code-density histogram

`AdcHistogram.java` bins every captured sample by its integer ADC code (2^{bitDepth} bins). For $\text{bitDepth} \leq 24$ a single `int[2bitDepth]` holds the counts; for 32-bit codes the 16 G-entry table is split into eight `int[229]` chunks addressed by a 3-bit selector + 29-bit offset (:49, :83). `Sample→code` masks the low `bitDepth` bits of the signed sample (:91); the class also tracks `uniqueCodes`, `min/maxCount` and `totalCount` as capture-quality metrics. `HistogramMode.java` records live audio for a set duration, skips the first ~100 ms of startup transient, and can export/replay the histogram as CSV (`code_unsigned;count`).

The **expected** per-code density (the ideal a real code count is compared against) comes from `WeightedBuffer.java` in two forms:

- **Moving-average reference** — $\text{weighted}[k] = \text{count}[k] / \text{movingAvg}[k]$, where `movingAvg` is a 12- or 25-tap **FIR** (`FIR.java`, direct-form tapped delay line) smoothing of the raw counts; the global average is recomputed only over codes whose weight lands in `[0.5, 2.0]` to drop rails and outliers (:130).
- **Sine (arcsine) PDF reference** — for a known sine stimulus the theoretical code density is the arcsine law $f(V) = 1 / (\pi \cdot \sqrt{A^2 - V^2})$ (:266). Amplitude `A` may be supplied or estimated by inverting the empirical CDF at quantiles $F \in [0.10, 0.40] \cup [0.60, 0.90]$ (avoiding rails and the singular median, :404); when $A > 1$ (clipping) the rail bins absorb the clipped fraction $(2/\pi) \cdot \arccos(1/A)$ (:246).

5.2 DNL / INL (code-density test)

`DnlInlExporter.java` turns the per-code weights into converter-linearity curves in LSB:

- **DNL** — $\text{DNL}[k] = \text{weighted}[k] - 1$ (a code wider than ideal counts fewer hits → positive DNL).
- **INL** — $\text{INL}[k] = \sum_{i \leq k} \text{DNL}[i]$, the running integral of DNL from code 0 (:99).

Both are computed in one pass and exported (`code_hex;code_unsigned;{dnl,inl}_lsb`); for charting the per-code values are bucket-averaged to one point per pixel, $X = \text{code} \cdot \text{scaleVolts} / \text{binCount}$, $Y = \text{LSB}$, with 10 % auto-margins (:72).

5.3 Regression sine-fit calibration

When histogram binning is too noisy or unavailable, `RegressionCalibrator.java` (`RegressCalibrateMode`) recovers the per-code error curve directly from the raw samples by least squares:

1. **Frequency** — FFT peak for a rough estimate, then a **golden-section search** minimizing residual sum of squares (:240, tested on $\leq 2^{16}$ samples for speed).
2. **Linear LS sine fit** — $y[n] = a \cdot \sin \omega n + b \cdot \cos \omega n + c$ via 3×3 normal equations over all samples; $A = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$, DC `c` (:289).
3. **Per-code error** — for each sample accumulate `sample - ideal (LSB)` into `errorSum[code]`; average per code, then linearly interpolate interior codes the signal never visited (:158, :198).
4. **Export** — smooth the per-code error with a Gaussian-weighted moving average ($\sigma = \sqrt{\text{codeCount}/2}$, radius 3σ) and write effective bin widths `voltPerLsb · (1 - ΔsmoothedError)` in

the same `weighted_scaled` format the histogram path emits, so both feed the same downstream linearization (`:364`).